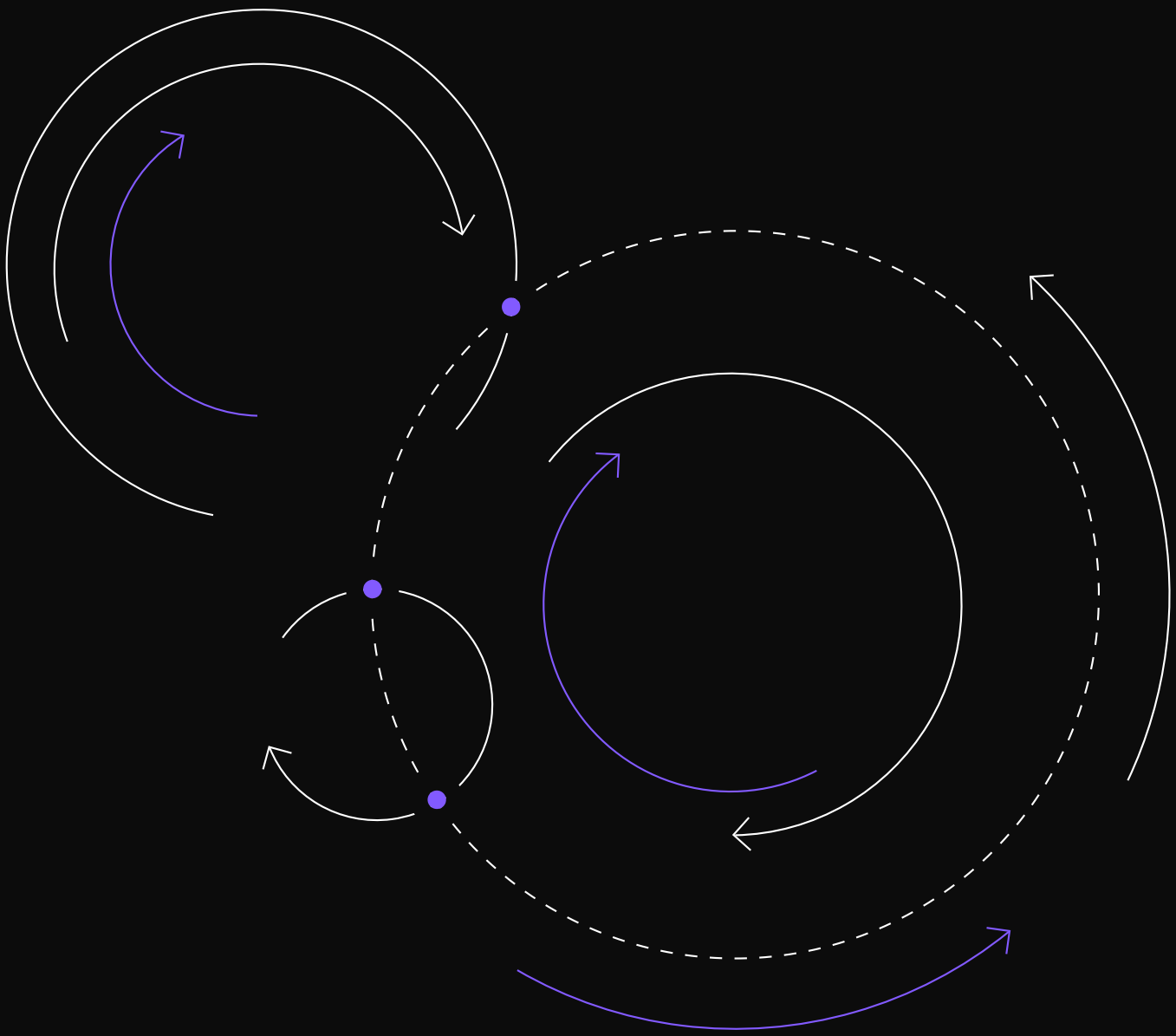
Машинное обучение

Лонгрид

Неделя 4

Линейная регрессия



Содержание

1. Что такое линейная модель 5

1.1. Основная формула линейной регрессии 5

2. Где применяется линейная регрессия 6

2.1. Прогнозирование спроса 7

2.2. Оценка влияния факторов на целевую переменную (эконометрическое применение) 8

3. Линейная регрессия от одной переменной 10

3.1. Функция ошибки RSS 11

3.2. Геометрическая интерпретация одномерного случая 12

3.3. Вывод параметров линейной регрессии от одной переменной 12

4. Линейная регрессия от нескольких переменных (общий случай) 13

4.1. Геометрическая интерпретация многомерного случая 15

5. Аналитическое решение для линейной регрессии в матричном виде 16

5.1. Псевдообратная матрица 17

6. Теорема Гаусса–Маркова 18

7. Предобработка признаков для линейных моделей 19

7.1. Binning 20

7.2. Log-transform 21

7.3. Polynomial Features 22

7.4. Преобразование таргета 23

7.5. Преобразования Бокса–Кокса и Йео–Джонсона 24

8. Итоги: постановка и решение задачи регрессии 25

9. Проблема вырожденности и ввод регуляризации 26

10. Регуляризация линейной регрессии 27

10.1. Идея регуляризации 28

11. Методы регуляризации 29

11.1. L_2-регуляризация 29

11.2. L_1-регуляризация 30

11.3. Elastic Net 31

11.4. Геометрическая интерпретация регуляризации 32

11.5. Аналитическое решение при использовании регуляризации 33

12. Другие решения вырожденности матрицы 34

12.1. Селекция признаков (Feature Selection) 34

12.2. Увеличение объёма выборки (Increasing Sample Size) 34

12.3. Использование устойчивых стохастических алгоритмов 35

13. Градиентный спуск для линейной регрессии 35

13.1. Почему градиентный спуск нужен для больших выборок 37

13.2. Есть ли риск попасть в локальный минимум 37

14. Реализации линейной регрессии: sklearn и statsmodels ... 37
14.1. scikit-learn: быстрое и гибкое обучение модели ... 38
14.1.1. Как считается решение линейной регрессии в sklearn ... 38
14.1.2. SVD-разложение (сингулярное разложение) ... 39
14.1.3. QR-разложение ... 39
14.1.4. Итеративные методы (градиентный спуск, SGD) ... 40
14.2. statsmodels: более глубокие выводы ... 41
14.2.1. Как работает OLS в statsmodels ... 42
15. Пример реализации линейной регрессии в statsmodels ... 46
15.1. Верхняя часть таблицы: общая информация ... 47
15.2. Метрики модели ... 48
15.3. Таблица коэффициентов ... 48
15.4. Пример интерпретации строки для признака x_1 ... 49
16. Какие библиотеки лучше использовать ... 49
16.1. scikit-learn ... 49
16.2. statsmodels ... 49
17. Если кратко ... 50

Контекст

Линейная регрессия — один из методов машинного обучения, который применяется для оценки зависимостей между признаками и целевой переменной при наличии линейной гипотезы. Линейная регрессия хорошо объясняет, как изменения признаков влияют на результат, и позволяет строить простые интерпретируемые модели.

Применение

Линейная регрессия используется для количественного анализа в экономике, бизнесе, науке и инженерии. С её помощью прогнозируют продажи, анализируют факторы спроса, оценивают влияние рекламы, исследуют взаимосвязи между медицинскими показателями, а также решают многие прикладные задачи, где требуется объяснить или предсказать числовой результат на основе набора признаков.

Краткий план

- → Что такое линейная модель
- → Где применяется линейная регрессия
- → Линейная регрессия от одной переменной
- → Линейная регрессия от нескольких переменных (общий случай)
- → Аналитическое решение для линейной регрессии в матричном виде
- → Теорема Гаусса–Маркова
- → Предобработка признаков для линейных моделей
- → Итоги: постановка и решение задачи регрессии
- → Проблема вырожденности и ввод регуляризации
- → Регуляризация линейной регрессии
- → Методы регуляризации
- → Другие решения вырожденности матрицы
- → Градиентный спуск для линейной регрессии
- → Реализации линейной регрессии: sklearn и statsmodels
- → Пример реализации линейной регрессии в statsmodels
- → Какие библиотеки лучше использовать
- → Если кратко

1. Что такое линейная модель

В машинном обучении линейными моделями называют алгоритмы, которые описывают зависимость между входными признаками и целевой переменной как линейную комбинацию признаков. Линейные модели — базовый инструмент анализа данных.

К этому классу относятся разные методы, которые используют похожий принцип, но решают разные задачи:

- **линейная регрессия** (англ. *linear regression*) — предсказывает числовые значения;
- **логистическая регрессия** (англ. *logistic regression*) — классифицирует объекты;
- **метод опорных векторов** (англ. *support vector machine,* SVM) — решает задачи классификации и регрессии с помощью поиска оптимальной разделяющей поверхности.

В этом лонгриде разберём линейную регрессию — базовую линейную модель, которая отвечает на вопрос «Сколько?»

С её помощью можно:

- предсказать стоимость квартиры по её параметрам;
- оценить будущий доход в зависимости от стажа и образования;
- спрогнозировать температуру воздуха по погодным признакам.

Линейная регрессия строит прогноз на основе простой идеи: итоговое значение выражается как сумма признаков, умноженных на свои коэффициенты. Эту зависимость описывает основная формула линейной регрессии.

1.1. ОСНОВНАЯ ФОРМУЛА ЛИНЕЙНОЙ РЕГРЕССИИ

Рассмотрим объект с набором признаков:

$$x = (x_1, x_2, \dots, x_d).$$

Нужно предсказать значение целевой переменной y. В линейной регрессии предполагается, что зависимость между y и признаками линейная и описывается как

$$y = w_0 + w_1 \cdot x_1 + w_2 \cdot x_2 + \cdots + w_d \cdot x_d + \epsilon,$$

где:

- w_0 — свободный член (англ. *intercept*);
- $w_1, w_2, \dots, w_d$ — веса признаков (коэффициенты модели);
- $x_1, \dots, x_d$ — значения признаков объекта;
- ϵ — ошибки, которые не удалось описать линейной формулой.

Формула описывает гиперплоскость, которая связывает признаки объекта и прогнозируемое значение y. Чем лучше модель подобрала коэффициенты, тем ближе прогноз к реальным данным.

На рис. 1 слева показан случай с одним признаком:

$$y = w_0 + w_1 x.$$

На рис. 1 справа показан случай с двумя признаками:

$$y = w_0 + w_1 \cdot x_1 + w_2 \cdot x_2.$$

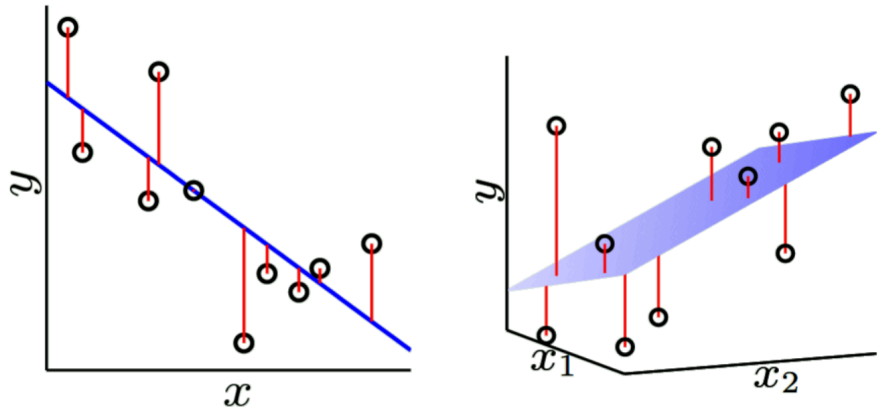


Рис. 1. Визуализация гиперплоскости в двумерном и трёхмерном пространствах. Источник

Чтобы записывать модель более компактно, перейдём к векторной форме:

$$\begin{aligned} y &= w_0 + w_1 x_1 + w_2 x_2 + \cdots + w_d x_d + \epsilon = \\ &= w_0 + \sum_{i=1}^{d} w_i x_i + \epsilon = \\ &= w_0 + \langle w, x \rangle + \epsilon, \end{aligned}$$

где:

- $x = (x_1, x_2, \dots, x_d)$ — вектор признаков;
- $w = (w_1, w_2, \dots, w_d)$ — вектор весов;
- $\langle w, x \rangle$ — скалярное произведение двух векторов;
- ϵ — ошибки.

2. Где применяется линейная регрессия

Линейная регрессия используется там, где нужно предсказать числовое значение на основе множества признаков. Она даёт не только прогноз, но и понятную интерпретацию результата. По коэффициентам модели можно понять, как каждый признак влияет на итоговое значение.

У линейной регрессии есть две основные задачи:

- предсказать число (например, стоимость квартиры, расход топлива);
- оценить влияние признаков на финальный результат, что особенно популярно в эконометрике.

ОПРЕДЕЛЕНИЕ

Эконометрика — наука, которая использует математические и статистические методы для анализа экономических явлений и взаимосвязей, а также для построения моделей, которые позволяют прогнозировать экономическое поведение и оценивать влияние различных факторов.

В эконометрике линейную регрессию часто используют для содержательных интерпретаций — например, чтобы оценить, как изменение одного признака связано с изменением результата. Для этого к модели добавляют предпосылки и доработки, например учитывают межвременные и пространственные эффекты.

Преимущества линейных моделей:

- легко обучаются даже на больших объёмах данных;
- хорошо интерпретируются (вес каждого признака можно объяснить);
- редко переобучаются при правильной подготовке данных (но переобучение всё равно возможно);
- являются базой для многих других моделей (например, линейных слоёв нейросетей).

Линейные модели часто применяются в задачах:

- где известна линейная зависимость между признаками и целевой переменной;
- между признаками отсутствует сильная корреляция, что снижает риск мультиколлинеарности;
- важна прозрачность модели и возможность объяснить вклад каждого признака в прогноз.

ОПРЕДЕЛЕНИЕ

Мультиколлинеарность — сильная корреляция между независимыми переменными в данных.

Ниже приведены несколько задач применения линейной регрессии, где чётко видно, где целевая переменная, а где входные признаки.

2.1. ПРОГНОЗИРОВАНИЕ СПРОСА

В интернет-магазинах нужно прогнозировать продажи конкретного товара, чтобы планировать закупки, рекламу и динамическое ценообразование.

Для этого часто используют воронку интереса: просмотр → клик → корзина → покупка.

Целевая переменная y (таргет): сколько штук товара купят за следующую неделю?

Признаки (описывают интерес к товару за текущую неделю):

- просмотры карточки,
- добавления в корзину,
- добавления в вишлист,
- поисковые запросы по товару,
- клики из рекомендательных блоков.

В этом случае линейная модель хорошо подходит для прогнозирования. Все признаки — это разные виды взаимодействия пользователя с товаром, и каждый из них может по-своему влиять на спрос. Благодаря этому прогноз линейной модели легко объяснить.

Просмотры (тыс.)	Корзина	Wishlist	Поиск	Рекомендации	Спрос (y)
5,0	40	60	80	140	110
5,5	50	40	100	100	111
6,0	80	60	90	120	116
6,5	90	40	110	80	117
7,0	100	60	80	140	120

Например, после оценки линейной модели получилось уравнение:

$$\text{спрос} = 1000 + 0{,}01 \cdot \text{просмотры} + 0{,}3 \cdot \text{вишлист} + 0{,}05 \cdot \text{рекомендации} + \ldots$$

Модель оценивает, сколько продаж связано с каждым типом действия:

- 1 просмотр $\rightarrow +0{,}01$ покупки;
- 1 добавление в вишлист $\rightarrow +0{,}3$ покупки;
- 1 клик из рекомендаций $\rightarrow +0{,}05$ покупки.

Линейная регрессия здесь подходит как базовая модель для быстрого прогноза. Более сложные модели могут дать более высокое качество, но их труднее интерпретировать (об интерпретации сложных моделей речь пойдёт на 13-й неделе).

Главное преимущество линейной модели в этом случае — её легко обучить и просто объяснить результат.

2.2. ОЦЕНКА ВЛИЯНИЯ ФАКТОРОВ НА ЦЕЛЕВУЮ ПЕРЕМЕННУЮ (ЭКОНОМЕТРИЧЕСКОЕ ПРИМЕНЕНИЕ)

Линейная регрессия — один из основных инструментов в эконометрике, где задача — не только получить точный прогноз, но и оценить влияние факторов и доверительные интервалы этих оценок.

Рассмотрим применение линейной регрессии на примере классического датасета Advertising из учебника *An Introduction to Statistical Learning* (James, Witten, Hastie, Tibshirani).

В нём собраны данные по 200 рынкам. Для каждого рынка известны расходы на разные рекламные каналы и объём продаж товара. Задача — понять, какие каналы действительно влияют на продажи.

Целевая переменная y: объём продаж товара на данном рынке (в тысячах единиц).

Признаки X:

- **TV** — бюджет на ТВ-рекламу (тысяч долларов);
- **Radio** — бюджет на радиорекламу (тысяч долларов);
- **Newspaper** — бюджет на газетную рекламу (тысяч долларов).

На рис. 2 показана связь между расходами на рекламу и продажами. По горизонтали отложены расходы на каждый канал рекламы, по вертикали — продажи. Красная линия — линия регрессии, показывающая средний тренд.

Рисунок 1. Связь между расходами на рекламу и объёмом продаж

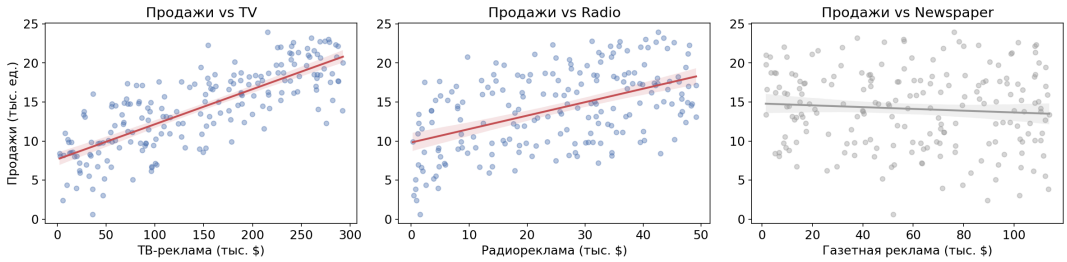


Рис. 2. Связь между расходами на рекламу и объемом продаж.Изображение создано автором

На графике видно: у ТВ и радио есть выраженная положительная связь с продажами — чем больше тратим, тем больше продаём. А вот для газет связь слабее: точки разбросаны, чёткого тренда нет.

Построим линейную регрессию:

$$\text{Sales} = w_0 + w_1 \cdot \text{TV} + w_2 \cdot \text{Radio} + w_3 \cdot \text{Newspaper}.$$

Модель объясняет 88% вариации продаж ($R^2 = 0{,}88$) — для линейной модели это высокий показатель. Но главное для эконометрики — не R^2, а коэффициенты при факторах и их значимость.

Результаты модели представлены на рис. 3.

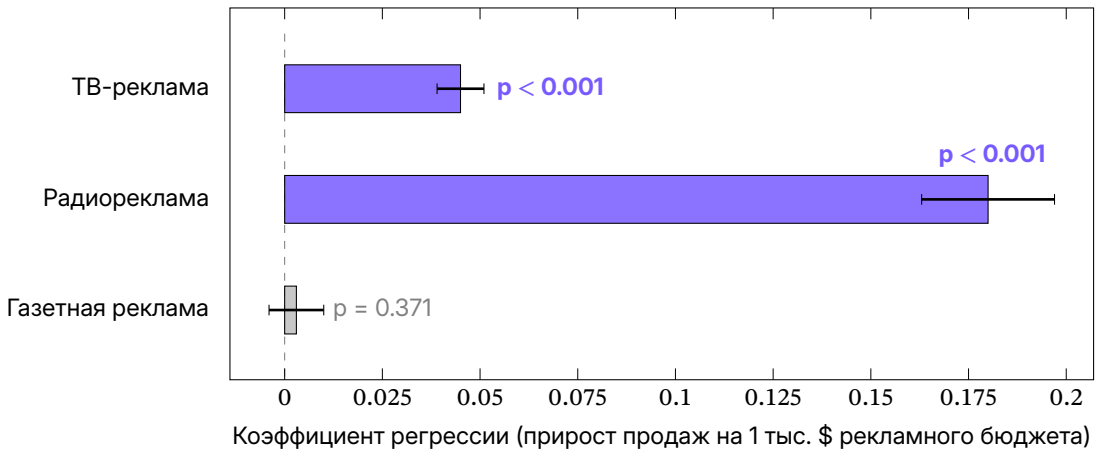


Рис. 3. Коэффициенты модели с 95%-ми доверительными интервалами. Изображение создано автором

На графике изображены коэффициенты линейной модели с их 95%-ми доверительными интервалами. Синие столбцы — статистически значимые факторы ($p < 0{,}05$), серый — незначимый. Коэффициент показывает, на сколько тысяч единиц вырастут продажи, если увеличить рекламный бюджет данного канала на 1 тысячу долларов.

ВАЖНО

По этой модели можно сделать содержательные выводы. Например:

«При прочих равных каждая дополнительная тысяча долларов, потраченная на радиорекламу, увеличивает продажи на 180 единиц. Для ТВ-рекламы эффект скромнее — плюс 46 единиц на тысячу долларов».

«При прочих равных» означает, что при анализе меняется только одна переменная за раз: увеличивается бюджет одного канала, а остальные остаются фиксированными.

Для газетной рекламы такой вывод сделать нельзя: доверительный интервал её коэффициента пересекает нулевую отметку ($p = 0{,}37$). Это значит, что связь между расходами на газеты и продажами статистически неотличима от нуля — модель не показывает значимого эффекта. В такой ситуации маркетолог может перераспределить часть бюджета с газет на радио и ТВ.

Далее разберём основные принципы построения и обучения линейной регрессии.

3. Линейная регрессия от одной переменной

Мы уже посмотрели, как линейную регрессию используют для прогнозирования и интерпретации факторов. Теперь разберём, как устроена сама модель и как она обучается.

Начнём с самого простого случая — модели с одним признаком. На нём проще понять геометрию линейной регрессии, смысл коэффициентов и принцип обучения модели.

У нас есть один признак x и одна целевая переменная y. Формула модели в этом случае:

$$y = w_0 + w_1 \cdot x.$$

Цель — подобрать такие коэффициенты w_0 и w_1, чтобы полученная прямая как можно точнее описывала зависимость между x и y.

Рассмотрим датасет о продажах в интернет-магазине.

Просмотры (тыс.) (x)	**Спрос (y)**
5,0	110
5,5	111
6,0	116
6,5	117
7,0	120
7,5	121
8,0	128
8,5	129
9,0	132

В таблице x — независимая переменная, y — целевая переменная, её также называют зависимой переменной, потому что её значение зависит от x.

Если построить точки (x_i, y_i) на плоскости, то задача линейной регрессии сводится к тому, чтобы провести через это облако точек прямую, которая будет в среднем ближе всего ко всем точкам.

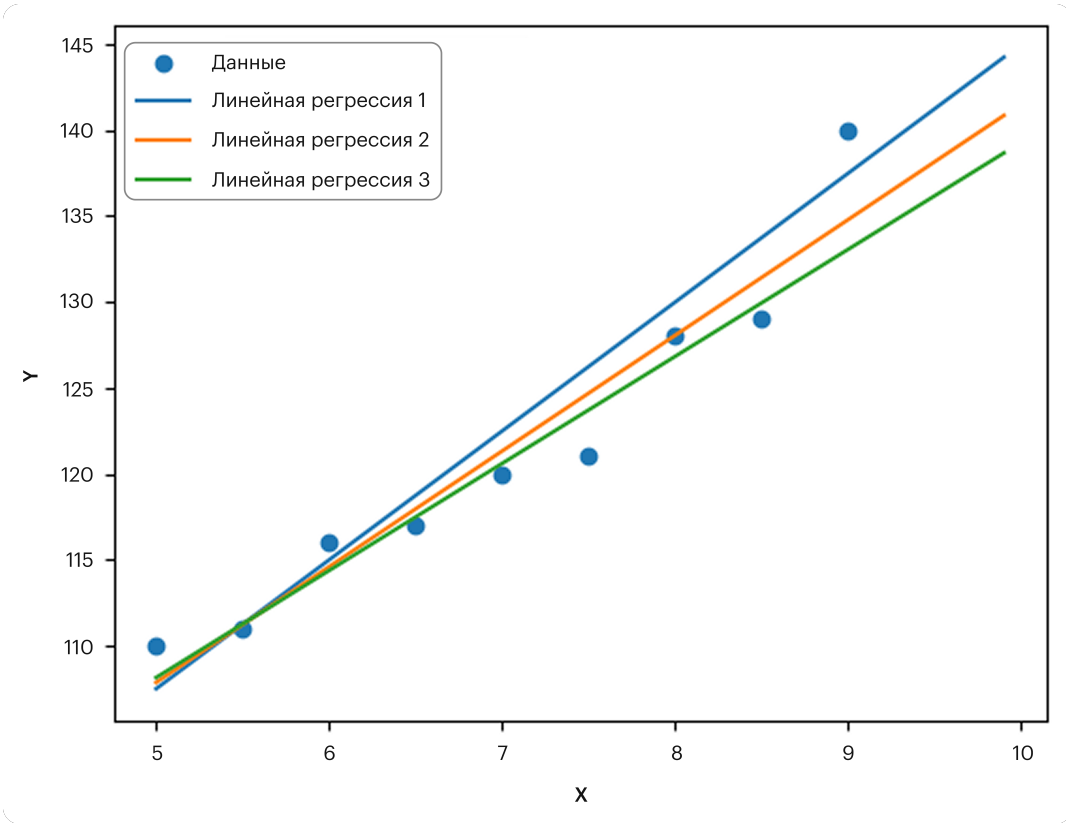


Рис. 4. Варианты проведения прямой через облако точек. Источник

3.1. ФУНКЦИЯ ОШИБКИ RSS

Задача обучения линейной регрессии — подобрать такие параметры, чтобы предсказания модели были как можно ближе к реальным значениям.

ОПРЕДЕЛЕНИЕ

RSS (англ. *residual sum of squares*) — это сумма квадратов остатков предсказания модели и истинного значения.

Нужно найти такие параметры w_0, w_1 (в общем случае — вектор w), при которых сумма квадратов отклонений на всей выборке будет минимальной. Параметры можно найти с помощью метода наименьших квадратов (МНК):

1. Для заданного набора весов расчёт идёт по каждому объекту i:
   - считаем предсказание $\hat{y}_i = w_0 + w_1 \cdot x_i$;
   - вычисляем отклонение $e_i = y_i - \hat{y}_i$;
   - берём квадрат этого отклонения e_i^2.
2. Суммируем квадраты по всей выборке и получаем функцию потерь:

$$\text{RSS} = \sum_i (y_i - \hat{y}_i)^2 = \sum_i \left(y_i - (w_0 + w_1 \cdot x_i)\right)^2.$$

   Как можно видеть, $\text{MSE} = \text{RSS}/n$.
3. Цель обучения — найти такие w, для которых RSS достигает своего минимума. Тогда среднее квадратическое отклонение между предсказанными и реальными значениями будет наименьшим.

Ниже разберём, как линейная регрессия обучается с точки зрения геометрии оптимизационной задачи.

Линейная регрессия минимизирует функцию потерь MSE (или RSS) — сумму квадратов отклонений между предсказанными и истинными значениями.

3.2. ГЕОМЕТРИЧЕСКАЯ ИНТЕРПРЕТАЦИЯ ОДНОМЕРНОГО СЛУЧАЯ

Если изобразить данные на плоскости, то координаты каждой точки — это (x_i, y_i). Модель строит прямую:

$$\hat{y} = w_0 + w_1 \cdot x.$$

Для каждого наблюдения (см. рис. 5) можно провести отрезок от точки наблюдения до прямой регрессии. Расстояние по вертикали между точкой и прямой $e_i = y_i - \hat{y}_i$ — остаток модели (англ. *residual*).

Квадрат длины этого отрезка — это ошибка (e_i^2) линейной регрессии на выбранном наблюдении или вклад в RSS. Чем ближе точки к прямой, тем меньше RSS и тем лучше модель.

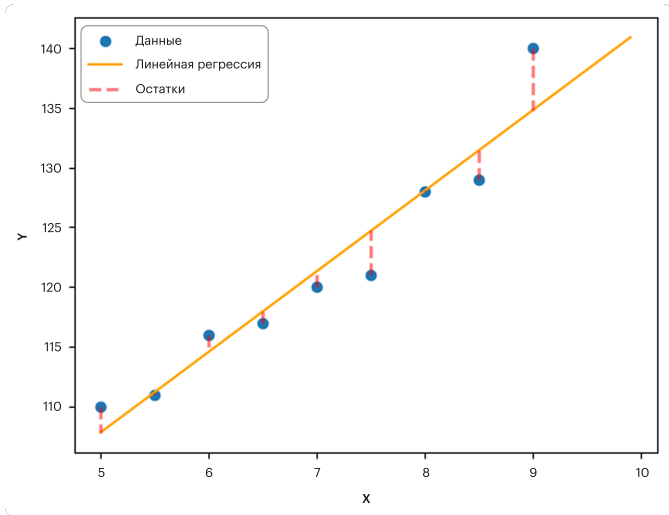


Рис. 5. Геометрическая интерпретация построения линейной регрессии. Изображение создано автором

3.3. ВЫВОД ПАРАМЕТРОВ ЛИНЕЙНОЙ РЕГРЕССИИ ОТ ОДНОЙ ПЕРЕМЕННОЙ

Нужно найти минимум RSS по параметрам модели. Данная функция дифференцируемая и выпуклая, поэтому, чтобы найти оптимальные значения параметров, берём производные по каждому параметру, приравниваем к нулю и выражаем аналитически:

$$\text{RSS} = \sum_i (y_i - \hat{y}_i)^2 = \sum_i \big(y_i - (w_0 + w_1 \cdot x_i)\big)^2 \to \min_{w_0, w_1}.$$

Тогда

$$\begin{cases} \dfrac{\partial\,\text{RSS}}{\partial w_0} = -2\sum_i (y_i - w_0 - w_1 \cdot x_i) = 0, \\ \dfrac{\partial\,\text{RSS}}{\partial w_1} = -2\sum_i (y_i - w_0 - w_1 \cdot x_i) x_i = 0. \end{cases}$$

Решение для w_0:

$$w_0 = \overline{y} - w_1 \overline{x},$$
$$\overline{y} = \frac{\sum_i y_i}{n},$$
$$\overline{x} = \frac{\sum_i x_i}{n}.$$

Подставим результат для w_0 во вторую производную. Получаем следующее:

$$w_1 = \frac{\left(\sum_i x_i y_i\right)/n - \overline{x}\,\overline{y}}{\left(\sum_i x_i^2\right)/n - \left[\left(\sum_i x_i\right)/n\right]^2} = \frac{\text{cov}(\{x_i\}, \{y_i\})}{\text{var}(\{x_i\})}.$$

Полученная прямая регрессии проходит через «центр масс» данных:

$$y - \overline{y} = \frac{\mathrm{cov}(\{x_i\},\{y_i\})}{\mathrm{var}(\{x_i\})}\,(x - \overline{x}).$$

Представь, что каждая точка — это маленький груз на невесомой доске. Центр масс — точка равновесия. Если подпереть доску ровно под центром масс, она не завалится. Прямая регрессии — это доска, которая всегда «балансирует» именно в этой точке.

Для доказательства достаточно подставить $x = \overline{x}$ в уравнение регрессии.

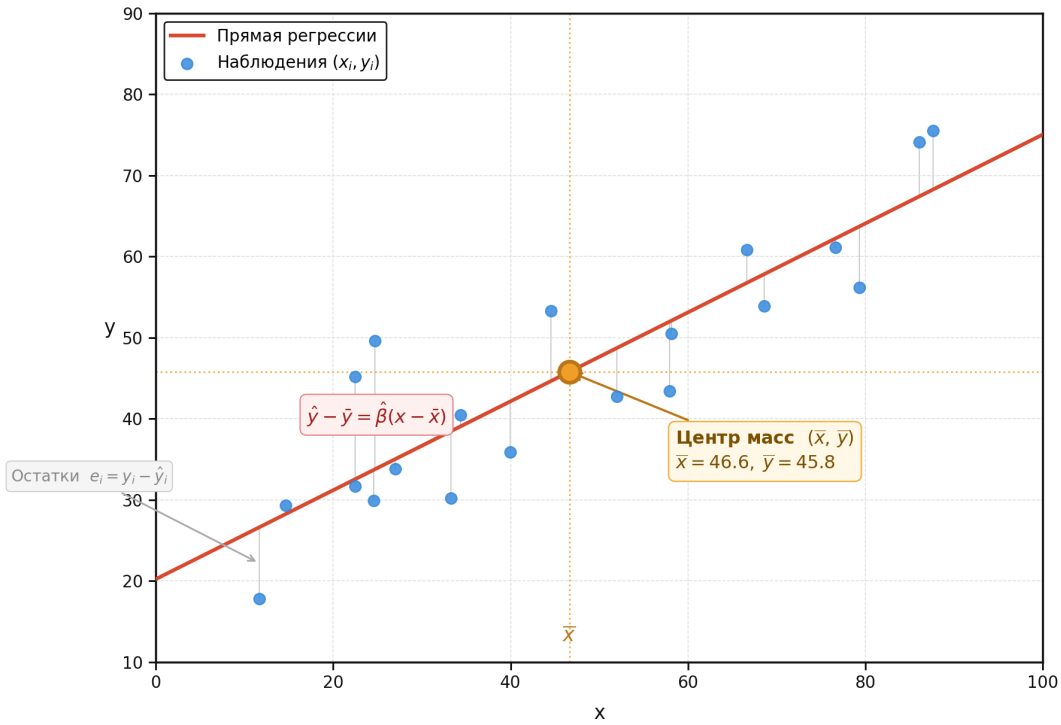


Рис. 6. Прямая регрессии всегда проходит через «центр масс». Источник

4. Линейная регрессия от нескольких переменных (общий случай)

Одного признака редко хватает для реальных задач. Например, спрос в интернет-магазине зависит не только от просмотров товара, но и от добавлений в корзину, рекомендаций, поиска и других факторов.

При переходе к нескольким признакам сама идея линейной регрессии не меняется: модель по-прежнему ищет линейную зависимость между признаками и целевой переменной. Но вместо прямой теперь строится гиперплоскость в многомерном пространстве.

Добавим в датасет продаж в интернет-магазине новые признаки.

Когда есть несколько признаков, линейная регрессия называется множественной и записывается так:

$$y = w_0 + w_1\,x_1 + w_2\,x_2 + \cdots + w_d\,x_d.$$

Просмотры (тыс.) (x_1)	Корзина (x_2)	Wishlist (x_3)	Поиск (x_4)	Рекомендации (x_5)	Спрос (y)
5,0	40	60	80	140	110
5,5	50	40	100	100	111
6,0	80	60	90	120	116
6,5	90	40	110	80	117
7,0	100	60	80	140	120
7,5	110	40	100	100	121
8,0	160	60	90	120	128
8,5	170	40	100	100	129
9,0	180	60	110	80	132

Чтобы записывать модель компактно, добавляют фиктивный признак $x_0 = 1$ и объединяют веса:

$$\tilde{x} = (1, x_1, x_2, \dots, x_d),$$
$$\tilde{w} = (w_0, w_1, w_2, \dots, w_d).$$

Тогда модель принимает следующий вид:

$$\hat{y} = \langle \tilde{x}, \tilde{w} \rangle .$$

Если нужно работать со всей обучающей выборкой из n объектов, удобно использовать матричную форму. Обозначим:

- X — матрица признаков размера $n \times (d+1)$, каждая строка — один объект $\tilde{x}$, где первый элемент всегда равен 1;
- y — вектор целевых значений длины n;
- w — вектор коэффициентов длины $d+1$ (d коэффициентов наклона и 1 коэффициент сдвига).

Тогда модель для всех объектов записывается следующим образом:

$$\hat{y} = Xw.$$

Как и в одномерном случае, цель регрессии — найти такие w, при которых предсказания $\hat{y} = X \cdot w$ будут как можно ближе к y. Формализуем это как задачу минимизации среднеквадратичной ошибки (MSE):

$$\|Xw - y\|^2 \to \min_w .$$

Здесь $\|\cdot\|^2$ — евклидова норма (сумма квадратов компонент вектора ошибки, тот же RSS или MSE).

4.1. ГЕОМЕТРИЧЕСКАЯ ИНТЕРПРЕТАЦИЯ МНОГОМЕРНОГО СЛУЧАЯ

Пусть каждый столбец матрицы X — это вектор длины n. Эти столбцы порождают некоторое подпространство в $\mathbb{R}^n$.

Цель — найти вектор $\hat{y} = Xw$:

- который лежит в этом подпространстве;
- как можно ближе к вектору y по евклидовой норме.

Такой вектор $\hat{y}$ — это ортогональная проекция вектора y на подпространство столбцов X. Модель проецирует y на пространство признаков (см. рис. 7).

В итоге:

- $\hat{y} = Xw$ — проекция;
- $e = y - \hat{y}$ — вектор ошибки, причём e перпендикулярен всем столбцам X.

Это означает, что $\langle X_j, e \rangle = 0$ для всех столбцов X_j — ошибка модели ортогональна пространству признаков.

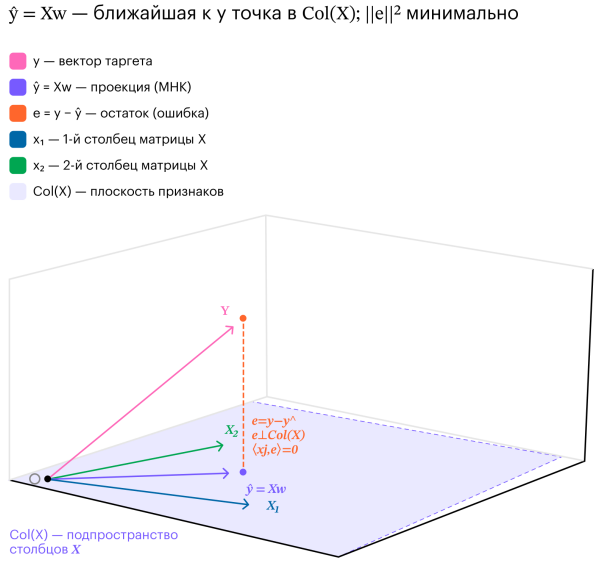


Рис. 7. МНК как ортогональная проекция. Изображение создано автором

Это ключевая геометрическая интерпретация линейной регрессии и метода наименьших квадратов: модель находит наилучшую в смысле нормы проекцию y на пространство, порождённое признаками.

Если признаки хорошо объясняют y, то проекция $\hat{y}$ почти совпадает с y, и ошибка e мала. Если признаки неинформативны или связь с целевой переменной нелинейна, то расстояние между y и $\hat{y}$ велико — модель плохо решает задачу.

Числовые признаки перед обучением линейной регрессии часто дополнительно преобразуют. Это помогает сделать модель устойчивее и повысить качество предсказаний.

- Нормализация данных помогает повысить стабильность и скорость сходимости, а также устойчивость решения.
- Добавление полиномов — линейные модели выделяют только линейные взаимосвязи. Полиномиальные признаки помогают модели учитывать нелинейные зависимости.
- Логарифмирование признаков экспоненциального вида помогает сделать зависимость ближе к линейной.
- В качестве функции для преобразования признаков можно использовать любую монотонную функцию без искажения зависимостей между признаком и целевой переменной.

5. Аналитическое решение для линейной регрессии в матричном виде

Запишем линейную регрессию в матричном виде. Пусть:

- X — матрица признаков размером $n \times (d+1)$, первая колонка заполнена единицами для свободного члена w_0 (см. рис. 8);
- w — вектор коэффициентов длины $d+1$: $(w_0, w_1, \dots, w_d)$;
- y — вектор истинных ответов длины n.

$$\begin{bmatrix} 1 & x_{11} & x_{12} & x_{13} & x_{14} \\ 1 & x_{21} & x_{22} & x_{23} & x_{24} \\ 1 & x_{31} & x_{32} & x_{33} & x_{34} \\ 1 & x_{41} & x_{42} & x_{43} & x_{44} \end{bmatrix} \times \begin{bmatrix} w_0 \\ w_1 \\ w_2 \\ w_3 \\ w_4 \end{bmatrix} = \begin{bmatrix} w_0 + x_{11}w_1 + x_{12}w_2 + x_{13}w_3 + x_{14}w_4 \\ w_0 + x_{21}w_1 + x_{22}w_2 + x_{23}w_3 + x_{24}w_4 \\ w_0 + x_{31}w_1 + x_{32}w_2 + x_{33}w_3 + x_{34}w_4 \\ w_0 + x_{41}w_1 + x_{42}w_2 + x_{43}w_3 + x_{44}w_4 \end{bmatrix} = \begin{bmatrix} y_1 \\ y_2 \\ y_3 \\ y_4 \end{bmatrix} + \begin{bmatrix} \varepsilon_1 \\ \varepsilon_2 \\ \varepsilon_3 \\ \varepsilon_4 \end{bmatrix}$$

Рис. 8. Постановка модели линейной регрессии в векторной форме. Изображение создано автором

Найдём аналитическое решение.

1. **Предсказания.** Вектор предсказаний $\hat{y}$ получается умножением X на w:

$$\hat{y} = Xw.$$

2. **Функция потерь (MSE).** Раскроем скобки:

$$J(w) = \|Xw - y\|^2 = (Xw - y)^\top (Xw - y) = \left(w^\top X^\top - y^\top\right)(Xw - y) = y^\top y - 2y^\top Xw + w^\top X^\top Xw.$$

3. **Градиент функции потерь:**

$$\operatorname{grad} J(w) = -2X^\top y + 2X^\top Xw = 2X^\top (Xw - y).$$

4. **Нормальное уравнение.** Чтобы найти минимум, приравниваем градиент к нулю:

$$X^\top (Xw - y) = 0.$$

Это эквивалентно

$$X^\top Xw = X^\top y.$$

5. **Аналитическое решение.** Если матрица $X^\top X$ обратима, то

$$w = (X^\top X)^{-1} X^\top y.$$

Получается, что обучение линейной регрессии сводится к решению системы линейных линейных уравнений $X^\top Xw = X^\top y$. После этого коэффициент w можно вычислить напрямую.
Ниже показано, как вычисляются коэффициенты на примере с тремя точками.

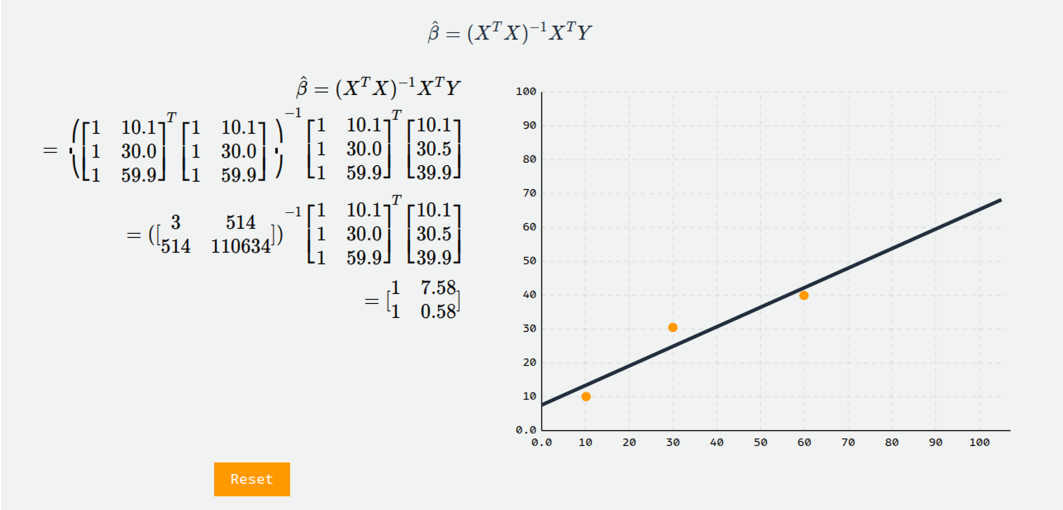


Рис. 9. Интерактивный пример. Источник

Составляющие формулы:

- $X^\mathsf{T} \cdot X$ — матрица попарных скалярных произведений признаков. Её ещё называют матрицей Грама.
- $X^\mathsf{T} \cdot y$ — вектор скалярных произведений каждого признака с целевой переменной.
- $\left(X^\mathsf{T} \cdot X\right)^{-1} \cdot X^\mathsf{T}$ — «левая псевдообратная матрица». Она позволяет выразить решение без итераций.

Если матрица $X^\mathsf{T} \cdot X$ необратима (например, из-за линейной зависимости между признаками), аналитическое решение нельзя получить напрямую. В этом случае используется регуляризация или сингулярное разложение.

На практике у решения $\left(X^\mathsf{T} X\right)^{-1} X^\mathsf{T}$ есть ограничения. Если признаки скоррелированы или их больше, чем наблюдений, обратная матрица может не существовать. В этом случае используют псевдообратную матрицу или численные методы.

5.1. ПСЕВДООБРАТНАЯ МАТРИЦА

Аналитическое решение линейной регрессии выглядит как

$$w = \left(X^\mathsf{T} \cdot X\right)^{-1} \cdot X^\mathsf{T} \cdot y.$$

Это решение корректно только в том случае, если матрица $X^\mathsf{T} \cdot X$ обратима. Однако если признаки линейно зависимы или почти зависимы, то $X^\mathsf{T} \cdot X$ становится вырожденной (сингулярной) или плохо обусловленной.

ОПРЕДЕЛЕНИЕ

Обусловленность — определение из численного анализа, которое показывает, насколько может измениться значение функции при небольшом изменении аргумента.

ОПРЕДЕЛЕНИЕ

Численный анализ — раздел математики, который занимается разработкой, анализом и реализацией алгоритмов для получения численных решений математических задач.

Плохо обусловленная матрица имеет большое число обусловленности. Это означает, что небольшие изменения в данных могут сильно менять результат вычислений.

Чем ниже обусловленность матрицы, тем устойчивее решение. И наоборот: высокая обусловленность делает модель чувствительной даже к небольшим ошибкам в данных.

В этом случае используется псевдообратная матрица — X^+ («икс плюс»). Тогда решение переписывается как

$$w = X^+ \cdot y.$$

Обычно псевдообратную матрицу вычисляют с помощью SVD.

Перед тем как перейти к практическим улучшениям модели (регуляризации и численным методам), убедимся, что статистические предположения соблюдены и понятны их последствия.

6. Теорема Гаусса–Маркова

Когда мы нашли аналитическое решение

$$w = \left(X^\top X\right)^{-1} X^\top y,$$

важно понять, насколько полученным коэффициентам можно доверять.

В реальных данных есть шум, измерительные ошибки и другие погрешности. Нужно гарантировать, что коэффициенты не просто хорошо описывают обучающие данные, а являются наилучшим выбором среди всех возможных простых линейных решений. В этом помогает теорема Гаусса–Маркова.

Теорема Гаусса–Маркова показывает, при каких условиях метод наименьших квадратов даёт оптимальные оценки коэффициентов.

Если выполняются следующие условия:

- модель правильно специфицирована, то есть мы добавили в неё все необходимые признаки;
- математическое ожидание ошибок равно нулю — $\mathbb{E}[\epsilon] = 0$;
- ошибки имеют одинаковую дисперсию — $\mathrm{var}(\epsilon_i) = \sigma^2$;
- ошибки не коррелированы между собой — $\mathrm{cov}(\epsilon_i, \epsilon_j) = 0$ при $i \neq j$;
- признаки не являются случайными величинами и не зависят от ошибок,

тогда оценка w, полученная с помощью линейной регрессии, является наилучшей несмещённой оценкой (*Best Linear Unbiased Estimator* — BLUE). Такие параметры имеют наименьшую дисперсию среди всех линейных несмещённых способов оценки.

ВАЖНО

Для того чтобы оценки коэффициентов были **несмещёнными**, достаточно, чтобы математическое ожидание ошибок было нулевым: $\mathbb{E}[\epsilon] = 0$.

7. Предобработка признаков для линейных моделей

Несмотря на простоту модели, она чувствительна к тому, как представлены данные. Чтобы модель работала корректно, нужно привести все признаки к числовому виду и к совместимым шкалам.

Выбор преобразования зависит от типа признака:

1. **Числовые признаки (Numeric Features).** Scaling (нормирование):
   - StandardScaler (z-score):
   $$x' = \frac{x - \mu}{\sigma},$$
   где:
     - μ — среднее для обучающей выборки;
     - σ — стандартное отклонения для обучающей выборки.
   - MinMaxScaler (0–1):
   $$x' = \frac{x - x_{\text{min}}}{x_{\text{max}} - x_{\text{min}}}.$$

   Масштабирование не обязательно для обычного OLS и не меняет его предсказаний: если заменить признак x на $x' = cx$, коэффициент при нём станет $w' = w/c$, а прогноз останется прежним.
   Оно важно для регуляризации, градиентных методов и численной устойчивости.
   При сопоставимом влиянии на таргет признак меньшего масштаба получает больший по модулю коэффициент, а признак большего масштаба — меньший. Например, если площадь квартиры измеряется в диапазоне 10–200 м², а число комнат — от 1 до 5, то у «комнат» коэффициент окажется в десятки раз больше — просто из-за масштаба, а не из-за большей важности признака.
   Поэтому размер коэффициентов сам по себе не отражает важность признаков: он подстраивается под их шкалу. Для регуляризации это критично — штраф $\lambda \sum_j w_j^2$ одинаково давит на все веса, а значит, сильнее всего бьёт по признаку с самым мелким масштабом. После приведения признаков к одной шкале штраф становится честным ко всем.

2. **Категориальные признаки (Categorical Features).**
   Линейная регрессия (как и любой ML-алгоритм) не умеет по умолчанию работать с категориальными переменными, так как категории часто представляются в виде названий, а модели работают с числами. Поэтому необходимо применить простые кодировки категориальных признаков:
   - One-Hot Encoding (OneHotEncoder): бинарные колонки для каждой категории; при необходимости — *drop = 'first'*, чтобы избежать линейной зависимости.
   - Ordinal Encoding (LabelEncoder) — только если категории имеют естественный порядок, например размер одежды:
   $$\text{S} < \text{M} < \text{L} < \text{XL}.$$

   Другие кодировки рассмотрим на следующих неделях.

3. **Смешанные / специальные случаи.**
   - Binning (дискретизация) — разбить числовой признак на интервалы и сделать для каждого отдельный dummy-признак, если зависимость явно кусочно-линейная.
   - Log-transform — для сильно скошенных распределений данных:
   $$x' = \log(x + 1).$$
   - Polynomial — если требуется добавить нелинейность в данные.

Далее рассмотрим эти техники обработки подробнее. Они нужны для того, чтобы линейная модель

смогла эффективно решать нелинейные задачи.

Хотя модель называется линейной из-за линейности по параметрам, это не значит, что она способна выявлять только линейные закономерности. Если обработать данные так, чтобы в них была информация о возможных нелинейностях, модель сможет их корректно выучить.

7.1. BINNING

Binning используют, когда зависимость в данных меняется после некоторого порога. До этого значения модель ведёт себя по одному сценарию, а после — по другому.

Например, нужно предсказать прочность бетона по соотношению воды и цемента. На рис. 10 показана такая зависимость.

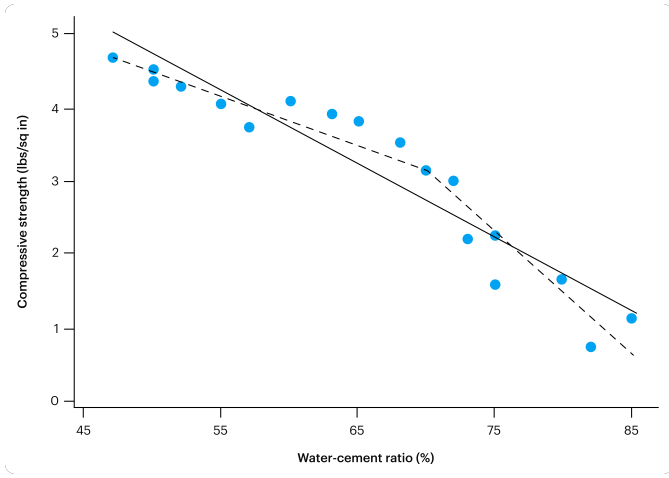


Рис. 10. Зависимость прочности бетона от соотношения воды и цемента. Сплошная линия — обычная линейная регрессия, пунктирная — кусочно-линейная модель. Источник

Обычная линейная регрессия (чёрная сплошная линия) даёт неплохое приближение, но модель можно улучшить, если перейти к кусочно-линейной зависимости (чёрная прерывистая линия). Для этого выбирают точку, после которой поведение зависимости меняется, и добавляют бинарный признак.

Предположим, такой точкой стало значение 70% для соотношения воды и цемента. Тогда уравнение линейной регрессии будет выглядеть так:

$$y = w_0 + w_1\, x_1 + w_2\, (x_1 - 70)\, x_2 + \epsilon,$$

где:

- y — прочность бетона;
- x_1 — соотношение воды и цемента;
- x_2 — бинарная переменная. Равна 0, если $x_1 \leq 70$; если иначе — равна 1.

Так модель получает информацию о смене режима зависимости и может описывать данные не одной прямой, а несколькими линейными участками.

7.2. LOG-TRANSFORM

Не все зависимости в данных линейны. Различные показатели растут со временем по экспоненте.

Например, если 10 лет расти по 10%, то итоговый рост составит примерно 260% ($(1{,}1)^{10}$), а не 100% ($1{,}1 \cdot 10$) — получаем экспоненциальный рост показателя.

Чтобы было возможно работать с такого рода зависимостями, обычно используют логарифмическое преобразование данных. Рассмотрим пример на рис. 11.

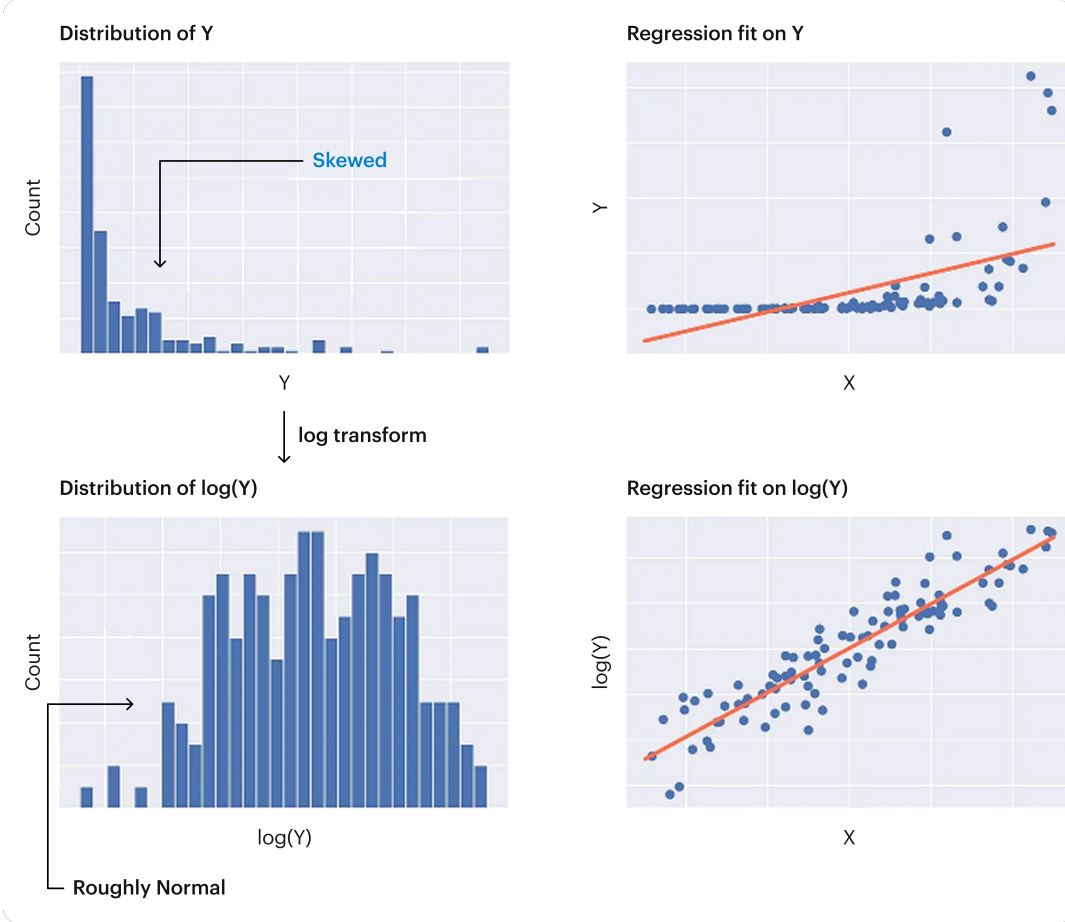


Рис. 11. Пример применения логарифмического преобразования с целью линеаризации данных. Источник

Изначально распределение смещено к малым значениям (см. рис. 11, левое верхнее изображение). На таких данных линейная модель работает хуже.

Линейные модели обычно работают стабильнее, если распределение признаков ближе к нормальному.

Логарифмирование помогает сделать зависимость ближе к линейной.

Было:

$$\hat{y} = w_0 + w_1 x_1.$$

Стало:

$$\hat{y} = w_0 + w_1 \log(x_1).$$

7.3. POLYNOMIAL FEATURES

Если у задачи сложная нелинейная гладкая зависимость (кусочно-линейные преобразования не помогают), можно создавать новые переменные с помощью функциональных преобразований исходных признаков.

Чаще всего для этого используют полиномиальные признаки. Например, если исходная линейная регрессия имеет вид

$$\hat{y} = w_0 + w_1 x_1,$$

после добавления полиномов до 3-й степени получаем

$$\hat{y} = w_0 + w_1\, x_1 + w_2\, x_1^2 + w_3\, x_1^3.$$

Теперь модель может учитывать кубические зависимости между признаками и целевой переменной. Однако если добавить больше полиномов для всех переменных, не получится решать любую задачу.

В модель также добавляются весовые коэффициенты. Чем их больше, тем модель становится выразительнее и всё больше подстраивается под данные, это приводит к переобучению.

Рассмотрим рис. 12.

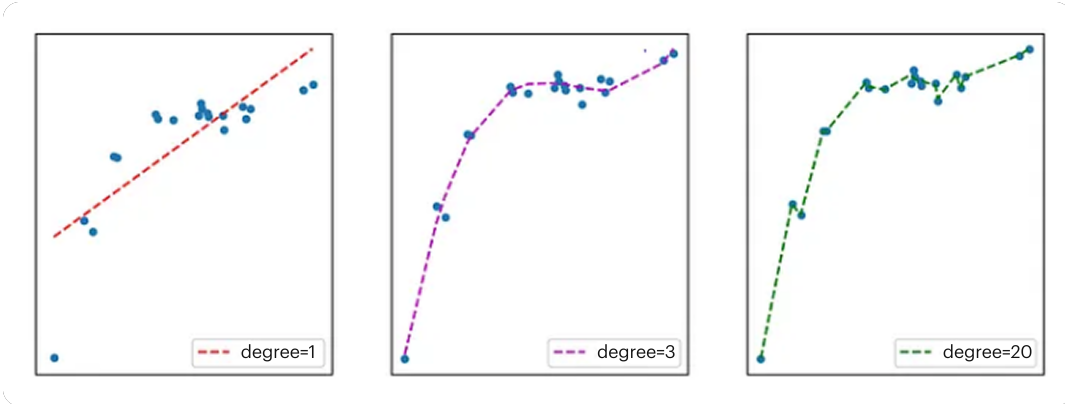


Рис. 12. Пример решения простой задачи с помощью полиномиальной регрессии. Источник

На левом рисунке обычная линейная регрессия плохо описывает зависимость. После добавления полиномиальных признаков до 3-й степени качество модели улучшается.

Затем в модель добавляют полиномы до 20-й степени. На правом рисунке модель переобучилась: кривая проходит почти через каждую точку обучающей выборки.

Такая модель обычно плохо работает на новых данных и даёт большую ошибку на тестовой выборке.

7.4. ПРЕОБРАЗОВАНИЕ ТАРГЕТА

Выше мы упоминали log-transform как способ обработки скошенных **признаков**. Но иногда логарифмировать нужно не X, а саму целевую переменную y.

Типичная ситуация: таргет — положительная величина с «тяжёлым правым хвостом» (зарплаты, цены, выручка, трафик). Если построить линейную регрессию на таких данных напрямую, на *residual plot* (ϵ vs $\hat{y}$) появляется характерный «веер» — дисперсия остатков растёт вместе с $\hat{y}$ (подробнее про *residual plot* описано в части «Как работает OLS в statsmodels»). Это **гетероскедастичность**: OLS-оценки коэффициентов остаются несмещёнными, но их стандартные ошибки занижены, а p-values и доверительные интервалы — ненадёжны.

Решение — перейти к модели:

$$\ln(y) = \beta_0 + \beta_1 x_1 + \cdots + \beta_p x_p + \epsilon.$$

Рассмотрим пример: предсказание зарплаты (Salary, тысяч рублей) по стажу работы (Experience, лет). На рисунке ниже — сравнение исходной и log-модели.

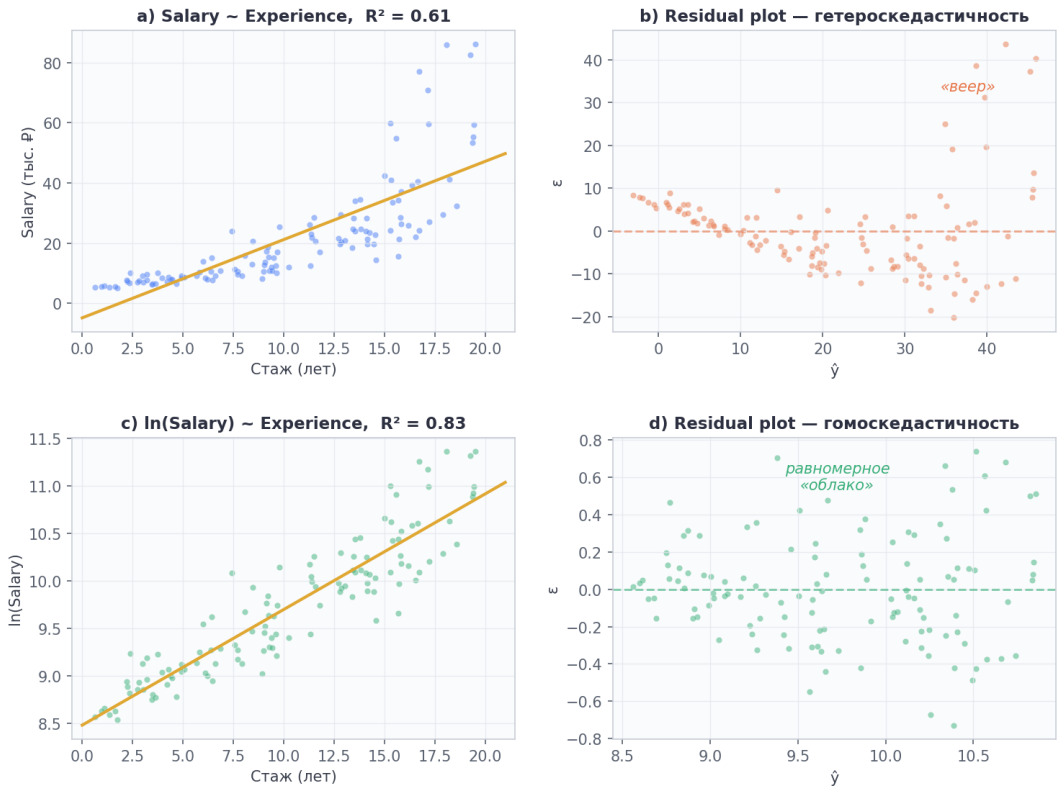


Рис. 13. Верхний ряд: модель на исходных данных и её residual plot с «веером». Нижний ряд: после логарифмирования таргета. Изображение создано автором

После логарифмирования меняется **интерпретация коэффициентов**: $\beta_1 \approx 0{,}122$ означает не «+0,122 тысяч рублей за год стажа», а «+12,2% к зарплате за каждый год». Формально:

$$\frac{\Delta y}{y} \approx \beta_1 \cdot \Delta x$$

— это **полуэластичность** (*semi-elasticity*).

Для денежных показателей процентное изменение часто интерпретировать удобнее, чем абсолютное.

7.5. ПРЕОБРАЗОВАНИЯ БОКСА–КОКСА И ЙЕО–ДЖОНСОНА

Тип преобразования зависит от формы распределения и характера зависимости между признаками.

- Если значения признака сильно смещены к малым значениям, а справа есть длинный хвост, часто применяют логарифмирование.
- Если зависимость X_2 от X_1 выпукла вниз, то следует повысить степень переменной X_2, перейдя от X_2 к $X_2^\lambda, \lambda > 1$. В случае выпуклости вверх — наоборот.
- Повышение степени увеличивает разброс в правой части значений, а понижение — в районе нуля.

Для таких случаев используют параметрические преобразования Бокса–Кокса и Йео–Джонсона. Они помогают сделать распределение признаков ближе к нормальному и упростить зависимость между признаками и таргетом.

Преобразование Бокса–Кокса применяют только к положительным значениям:

$$g_{\text{BC},\lambda}(x) = \begin{cases} \frac{x^\lambda - 1}{\lambda}, & \lambda \neq 0, \\ \ln(x), & \lambda = 0. \end{cases}$$

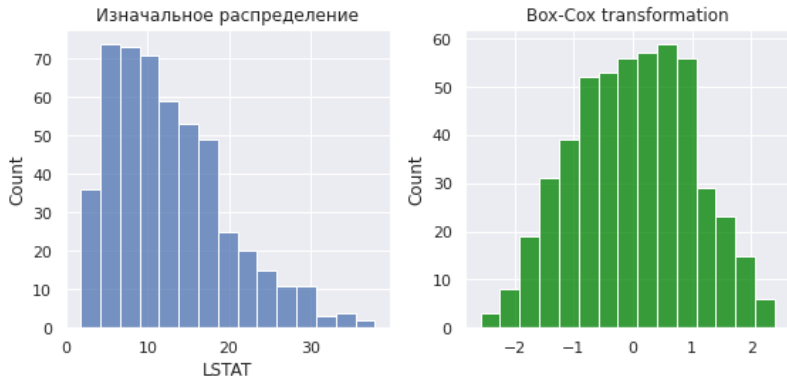


Рис. 14. Преобразование Бокса–Кокса до и после. Источник

Это работает для положительной переменной.

В случае произвольных знаков применяют преобразование Йео–Джонсона (`scipy.stats.yeojohnson`):

$$g_{\text{YJ},\lambda}(x) = \begin{cases} \frac{(1+x)^\lambda - 1}{\lambda}, & \lambda \neq 0, \quad x \geq 0, \\ \ln(x+1), & \lambda = 0, \quad x \geq 0, \\ -\frac{(1-x)^{2-\lambda} - 1}{2-\lambda}, & \lambda \neq 2, \quad x < 0, \\ -\ln(1-x), & \lambda = 2, \quad x < 0. \end{cases}$$

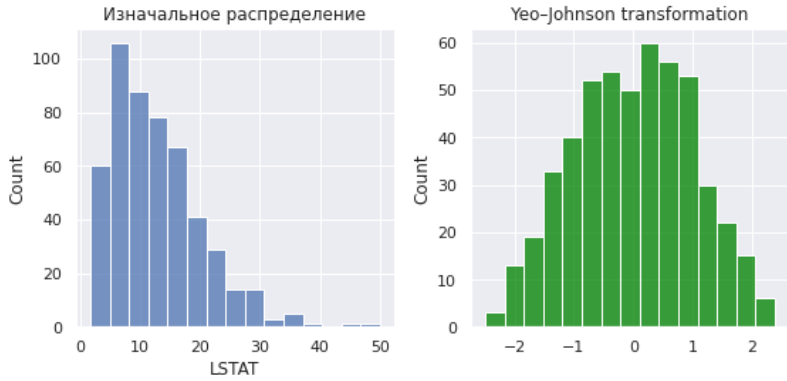


Рис. 15. Преобразование Йео–Джонсона до и после. Источник

Числовые признаки для линейной регрессии предобрабатывают по нескольким причинам:

- Нормализация данных помогает повысить стабильность обучения, ускорить сходимость и сделать решение устойчивее.
- Добавление полиномов — линейные модели могут выделять в данных только линейные взаимосвязи, но если сами данные будут нелинейные, то модель сможет выделить нелинейные зависимости как линейную комбинацию нелинейных данных. Полиномы позволяют добавить нелинейные признаки.
- Логарифмирование признаков помогает сделать экспоненциальные зависимости ближе к линейным и улучшить качество модели.
- В качестве функции для преобразования признаков можно использовать любую монотонную функцию без искажения зависимостей между признаком и целевой переменной.

8. Итоги: постановка и решение задачи регрессии

Мы прошли полный путь от постановки задачи до готовой к применению модели.

Постановка задачи. Линейная регрессия предполагает, что целевая переменная y линейно зависит от признаков X: модель ищет гиперплоскость, которая наилучшим образом приближает данные. Критерий качества — минимизация суммы квадратов отклонений (RSS).

Аналитическое решение. При выполнении условий Гаусса–Маркова метод наименьших квадратов даёт единственную оптимальную оценку коэффициентов:

$$w = (X^\mathsf{T} X)^{-1} X^\mathsf{T} y.$$

Эта формула работает, когда матрица $X^\mathsf{T}X$ обратима. Если нет — задача решается через псевдообратную матрицу, вычисляемую с помощью SVD.

Предобработка данных. Для того чтобы модель работала корректно, признаки нужно привести к совместимым шкалам (*scaling*), правильно закодировать (*encoding*) и при необходимости обогатить нелинейными преобразованиями — бинаризацией, полиномами, логарифмированием. Всё это не меняет природу модели, но существенно влияет на качество её оценок.

Однако даже идеально предобработанные данные не гарантируют устойчивости решения: матрица $X^\top X$ может оказаться плохо обусловленной или вырожденной. В таких случаях одной предобработки недостаточно — нужен дополнительный механизм управления сложностью модели. Именно о нём пойдёт речь дальше.

9. Проблема вырожденности и ввод регуляризации

Метод наименьших квадратов хорошо работает в теории, но на практике данные редко соответствуют всем предпосылкам модели. Из-за этого линейная регрессия может становиться нестабильной и переобучаться.

В таких случаях нужен дополнительный механизм управления сложностью модели — регуляризация. Сначала рассмотрим, какие проблемы возникают без неё.

- **Мультиколлинеарность.** Некоторые признаки почти линейно зависимы, из-за чего матрица $X^\top X$ становится плохо обусловленной, а коэффициенты модели — нестабильными.
  Например, в модели есть два признака: доход в рублях x_1 и доход в тысячах рублей $x_2 = x_1/1000$. Это фактически один и тот же признак.
  Модель может распределить вклад между ними разными способами. Например:

  $$w_1 = 1\,001\,000, \qquad w_2 = -1000.$$

  Предсказание останется тем же, но коэффициенты станут очень большими и чувствительными к изменениям данных. Даже одно новое наблюдение может сильно изменить их значения.
  Поэтому стоит провести чистку данных, убрав часть таких предикторов.
- **Преобладание признаков над наблюдениями.** Когда количество признаков близко или больше количества наблюдений, матрица $X^\top X$ плохо обусловлена.
  При этом удаление признаков может привести к потере полезной информации.
- **Шум и выбросы.** Наличие сильных выбросов может изменять коэффициенты линейной регрессии. Рассмотрим рис. 16.

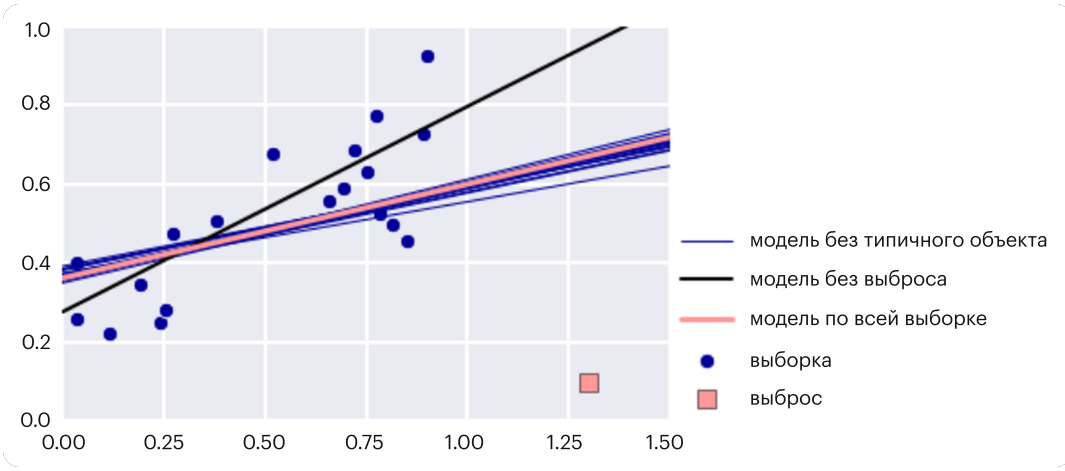


Рис. 16. Как выброс может повлиять на результаты работы всей модели. Источник

На рисунке видно, что даже один выброс может сильно изменить положение линии регрессии. Это связано с квадратичной функцией ошибки: большие отклонения увеличивают ошибку особенно сильно, поэтому выбросы заметно влияют на модель.

Если матрицу $X^\top X$ нельзя обратить или коэффициенты модели становятся нестабильными, используют регуляризацию — дополнительный штраф за большие значения весов.

Регуляризация помогает сделать модель устойчивее и снижает риск переобучения.

10. Регуляризация линейной регрессии

ОПРЕДЕЛЕНИЕ

Регуляризация — это способ заставить модель вести себя устойчиво, накладывая штраф за сложность модели. В линейной регрессии штраф накладывается на размер коэффициентов.

При обучении линейной регрессии обычно строится модель $y = X \cdot w$, а коэффициенты находятся по формуле $w = \left(X^\top \cdot X\right)^{-1} \cdot X^\top \cdot y$. Даже если получается хорошее приближение на обучающей выборке, могут возникнуть проблемы:

- предсказания на новых данных резко отличаются от реальных значений;
- некоторые веса w очень большие по модулю;
- небольшое изменение в выборке сильно меняет результат.

Это симптомы переобучения и численной нестабильности. Они особенно часто возникают, если:

- признаки сильно коррелированы между собой;
- в выборке мало объектов (данных мало, переменных много);
- есть шум в данных;
- в матрице X есть признаки, которые почти не несут информации.

В таких случаях линейная регрессия стремится найти коэффициенты, которые точно подстраиваются под каждый объект. Если данных мало или признаки шумные, коэффициенты могут становиться очень большими по модулю. Из-за этого качество модели на новых данных ухудшается.

10.1. ИДЕЯ РЕГУЛЯРИЗАЦИИ

Регуляризация помогает не только хорошо приближать y, но и избегать слишком больших весов модели. Для этого функцию ошибки дополняют штрафом за сложность модели (слишком большие коэффициенты).

Вместо стандартной функции потерь

$$\|Xw - y\|^2 \to \min_w$$

используют функцию с дополнительным слагаемым:

$$\Big(\|Xw - y\|^2 + \lambda R(w)\Big) \to \min_w,$$

где:

- $R(w)$ — функция регуляризации от значений весов;
- $\lambda \geq 0$ — гиперпараметр, который определяет силу штрафа.

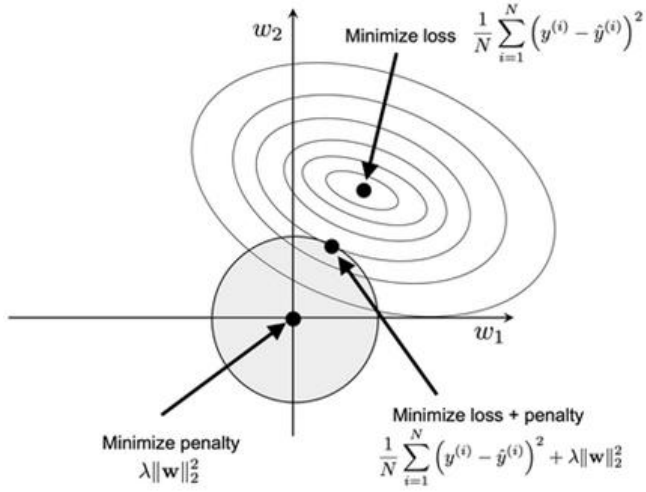


Рис. 17. Регуляризация добавляет штраф за большие коэффициенты и смещает решение к более устойчивым значениям весов. Источник

Теперь решение должно учитывать два требования:

- модель должна хорошо приближать данные (первая часть функции);
- модель не должна иметь слишком больших коэффициентов (вторая часть).

Баланс между этими требованиями задаёт параметр λ:

- если λ маленький, регуляризация почти не влияет на модель и коэффициенты будут близки к обычной линейной регрессии;
- если λ увеличивается, штраф за большие коэффициенты становится сильнее и веса начинают «сжиматься»;
- если λ слишком большой, модель начинает недообучаться и хуже решает задачу.

Поэтому гиперпараметр регуляризации нужно подбирать аккуратно.

11. Методы регуляризации

Существует несколько популярных видов регуляризации. Они различаются тем, какой штраф накладывается на коэффициенты модели и как это влияет на веса признаков.

11.1. L_2-РЕГУЛЯРИЗАЦИЯ

Этот метод добавляет штраф за большие значения коэффициентов. Чем больше веса модели, тем больше штраф.

> L_2-регуляризацию также называют **Ridge-регрессией**, или **гребневой регрессией** (*ridge* — «гребень»). Такое название связано с формой поверхности функции потерь после добавления штрафа. В математике этот подход также известен как **регуляризация Тихонова** — по фамилии математика Андрея Тихонова, который исследовал методы решения неустойчивых задач.

Формула L_2-регуляризации имеет вид

$$R(w) = \|w\|^2 = \sum_j w_j^2.$$

Тогда функция потерь записывается как

$$\|Xw - y\|^2 + \lambda \|w\|^2 \to \min_w.$$

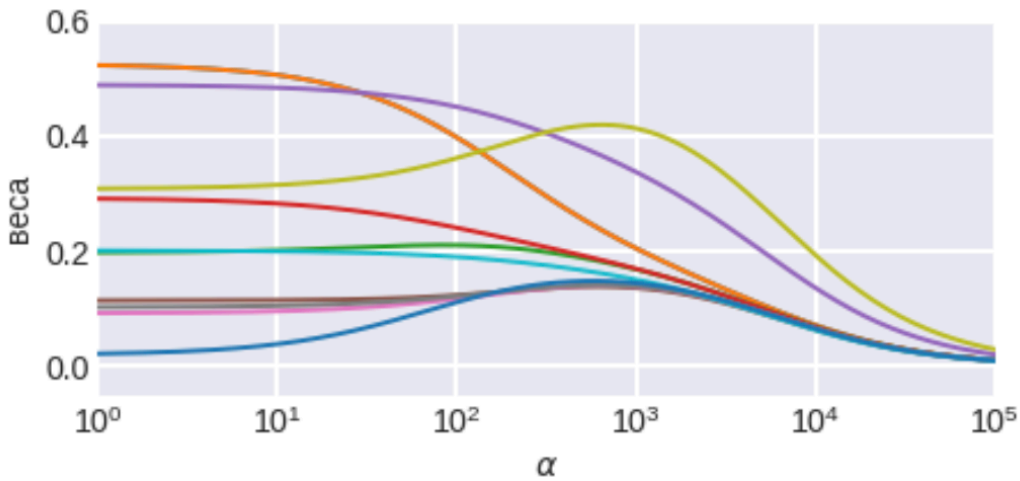


Рис. 18. Влияние коэффициента L_2-регуляризации на веса модели. Источник

Особенности L_2-регуляризации:

- все веса стремятся стать меньше по модулю, но остаются $\neq 0$;
- признаки не исключаются из модели полностью, а просто становятся менее важными (размер веса можно интерпретировать как важность признака);
- в отличие от LASSO, L_2 редко обнуляет коэффициенты полностью.

Иногда L_2-регуляризацию записывают в другой форме — как ограничение на размер весов:

$$\begin{cases} \|Xw - y\|_2^2 \to \min, \\ \|w\|_2^2 \le \lambda. \end{cases}$$

Это также L_2-регуляризация (по Иванову), но в другой постановке: она может быть сведена к регуляризации по Тихонову через лагранжиан.

11.2. L_1-РЕГУЛЯРИЗАЦИЯ

L_1-регуляризация, или LASSO (англ. *least absolute shrinkage and selection operator*), не только уменьшает веса модели, но и может автоматически исключать признаки из модели.

L_1-регуляризация вводит штраф за сумму абсолютных значений коэффициентов:

$$R(w) = \|w\|_1 = \sum_j |w_j| .$$

В отличие от L_2-регуляризации, LASSO может занулять отдельные коэффициенты.

Функция потерь имеет вид

$$\|Xw - y\|^2 + \lambda \|w\|_1 \to \min_w .$$

L_1-регуляризация стремится уменьшить веса, а некоторые из них делает равными нулю.

В результате модель автоматически отбирает признаки: переменные с нулевыми коэффициентами перестают влиять на предсказание.

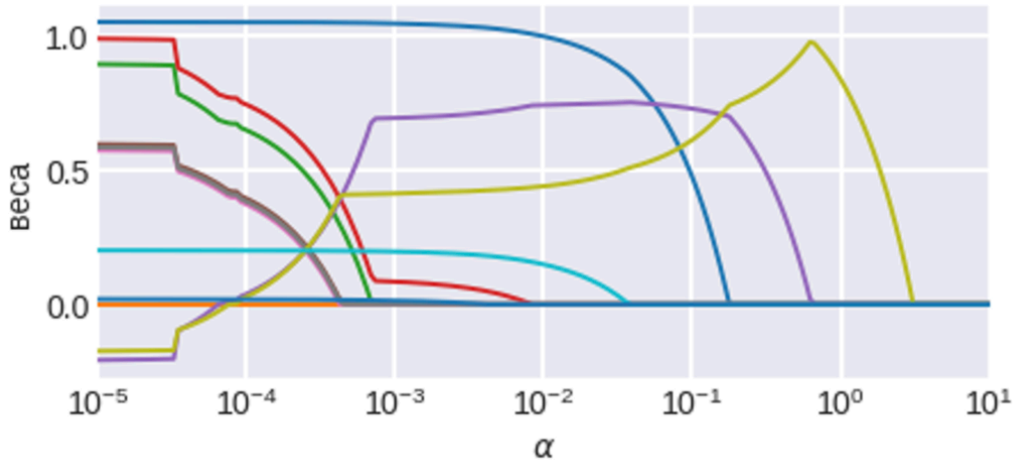


Рис. 19. Влияние коэффициента L_1-регуляризации на веса модели. Источник

Особенности L_1-регуляризации:

- хорошо работает, когда есть много неинформативных признаков, которые можно исключить;
- помогает бороться с мультиколлинеарностью — модель сама оставляет один из нескольких коррелированных признаков.

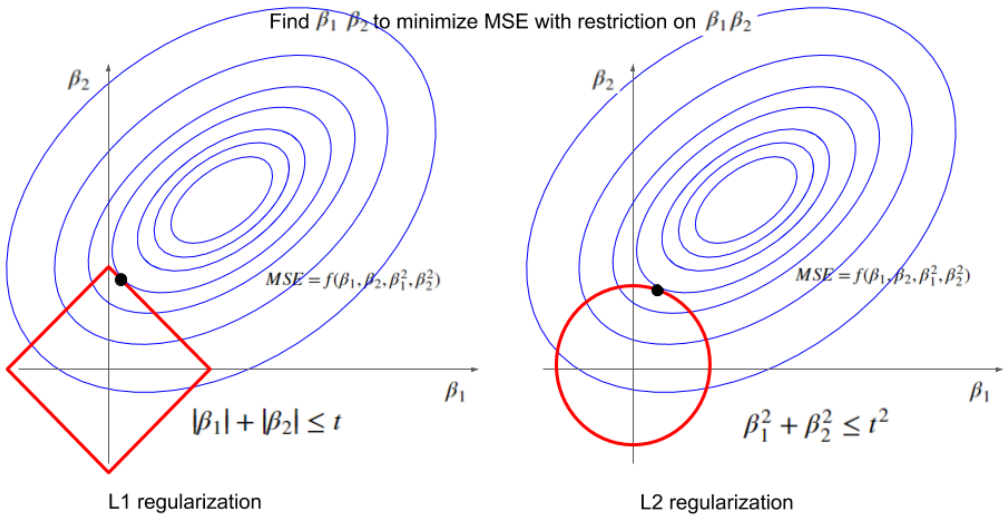


Рис. 20. Геометрическая интерпретация L_1- и L_2-регуляризации: LASSO может занулять коэффициенты за счёт ромбовидной формы ограничения, а Ridge-регрессия плавно сжимает веса. Источник

11.3. ELASTIC NET

Elastic Net сочетает свойства L_1- и L_2-регуляризации. Штраф выглядит следующим образом:

$$R(w) = \lambda_1 \|w\|_1 + \lambda_2 \|w\|^2.$$

Такое сочетание позволяет объединить преимущества обоих подходов:

- L_2-регуляризация помогает стабилизировать веса и распределять влияние между коррелированными признаками.
- L_1-регуляризация может занулять неинформативные признаки.
- На практике редко бывает так, что можно без потери качества исключить сразу много признаков из модели — обычно только небольшую часть; комбинация L_1- и L_2-штрафов помогает это контролировать.
- Elastic Net особенно полезен в задачах с большим количеством признаков и небольшой выборкой.

Elastic Net настраивается через два параметра: λ_1 и λ_2 или через λ и долю L_1-штрафа (обычно в `sklearn` это параметр `l1_ratio`).

11.4. ГЕОМЕТРИЧЕСКАЯ ИНТЕРПРЕТАЦИЯ РЕГУЛЯРИЗАЦИИ

Без регуляризации линейная регрессия ищет минимум функции ошибки $\|Xw - y\|^2$.
Геометрически это выглядит как семейство эллиптических линий уровня.

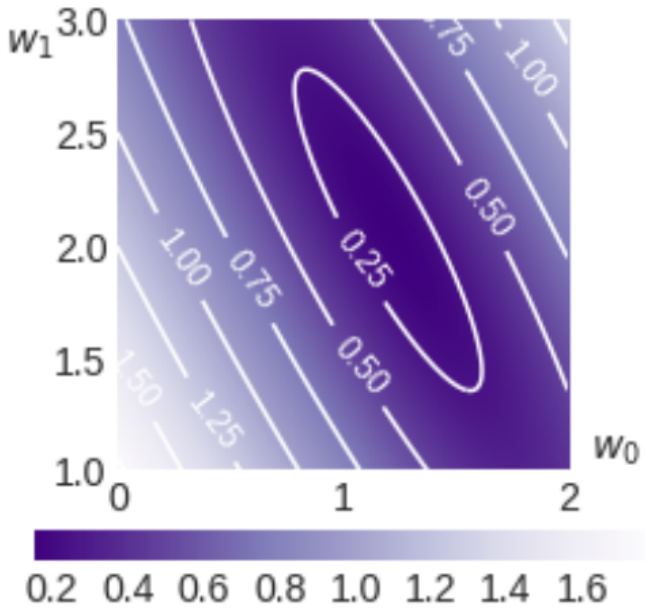


Рис. 21. Визуализация линий уровня функции потерь для линейной регрессии. Источник

ОПРЕДЕЛЕНИЕ

Линия уровня — геометрическое место точек (в нашем случае это параметры) плоскости, в которых функция принимает одно и то же значение.

Добавляя штраф $\lambda |w|^2$, мы «накладываем» на это пространство ещё одни, уже «круговые», линии уровня, исходящие от центра. В итоге минимум находится там, где растущий эллипс ошибки впервые соприкасается с кругом штрафа.

Чем больше λ, тем сильнее «тянет» к более короткому вектору w.

L_1-регуляризация создаёт **ромбовидное ограничение** ($\|w\|_1 \leq \text{const}$). У этого ограничения есть «углы» — точки, где один из коэффициентов равен нулю. Поэтому при минимизации функции потерь модель часто останавливается в этих углах.

Если нарисовать линии уровня функции ошибки (эллипсы):

- при L_2-регуляризации касание обычно происходит на границе круга, поэтому коэффициенты уменьшаются, но редко становятся равными нулю;
- **при L_1-регуляризации касание часто происходит в вершинах ромба, где часть коэффициентов равна нулю.**

Получить точку соприкосновения линии уровня функции потерь и штрафа в углу ромба «проще», так как можно провести через данный угол целое пространство касательных, в то время как при круглом штрафе возможна только одна касательная.

ВАЖНО

Пересечение линии уровня функции и штрафа объясняет ключевую разницу между методами регуляризации. L_2 делает веса маленькими, но не нулевыми. L_1 делает часть весов равными нулю.

На рис. 22 показано, как выглядят ограничения L_1, L_2 и Elastic Net в пространстве коэффициентов. Пересечения с линиями уровня функции ошибки происходят либо на окружности (L_2), либо в вершинах ромба (L_1). Elastic Net выглядит как ромб с закруглёнными сторонами. Такой подход позволяет уменьшить пространство возможных касательных для достижения нулевых весов.

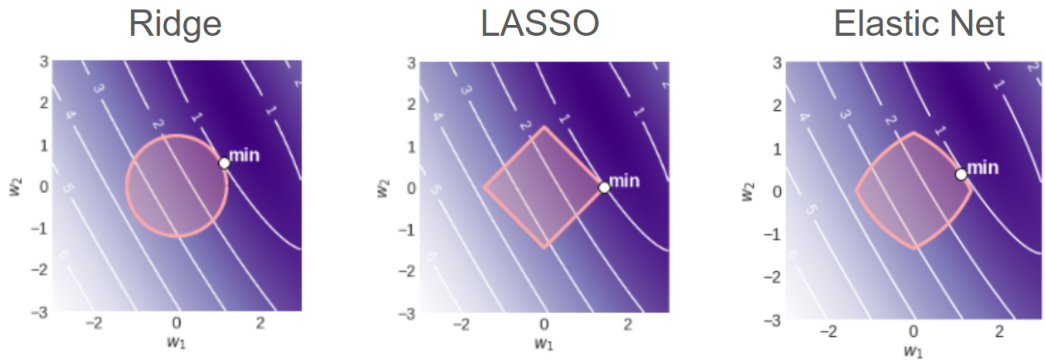


Рис. 22. Визуализация различных штрафов. Источник

В L_1-регуляризации мы пересекаемся с ромбом на $w_1 = 1{,}7$ или на $w_2 = 0$ — зануляем один из коэффициентов. В двух других вариантах штрафа зануление не произошло.

Регуляризация добавляет штраф за сложность модели — слагаемое к функции потерь, которое «тянет» коэффициенты ближе к нулю. Геометрически это означает, что минимум уже не ищется по всему пространству весов свободно: допустимая область ограничена, и решение находится там, где растущий эллипс ошибки впервые касается этой области — то есть при меньших по модулю коэффициентах, чем без штрафа.

За это приходится платить: ошибка на обучающей выборке чуть вырастет, потому что истинный минимум функции потерь лежит за пределами допустимой области. Но взамен модель перестаёт подстраиваться под каждое конкретное наблюдение и начинает лучше обобщаться на новых данных.

Виды регуляризации отличаются формой этой допустимой области и, как следствие, поведением коэффициентов:

1. L_2 **(Ridge)** — круговое ограничение; все коэффициенты равномерно сжимаются, но остаются ненулевыми.
2. L_1 **(LASSO)** — ромбовидное ограничение; у ромба есть углы на осях координат, поэтому решение часто попадает именно туда — часть коэффициентов обнуляется, модель автоматически отбирает признаки.
3. **Elastic Net** — комбинация обоих ограничений: сжимает веса, как Ridge, и обнуляет лишние, как LASSO.

11.5. АНАЛИТИЧЕСКОЕ РЕШЕНИЕ ПРИ ИСПОЛЬЗОВАНИИ РЕГУЛЯРИЗАЦИИ

Когда мы добавляем регуляризацию, это влияет на возможность получения аналитического решения вида

$$w = (X^\top X)^{-1} X^\top y.$$

Для L_2**-регуляризации** аналитическое решение сохраняется.

Функция потерь остаётся квадратичной:

$$\|Xw - y\|^2 + \lambda \|w\|^2 \to \min_w.$$

Её минимум можно найти аналитически:

$$w = (X^\top X + \lambda I)^{-1} X^\top y.$$

I — единичная матрица размерности $d + 1$. Это решение устойчиво даже при вырожденности $X^\top X$, потому что добавление λI гарантирует обратимость матрицы.

Для L_1**-регуляризации (LASSO)** аналитическое решение получить невозможно. Функция потерь $\|X \cdot w - y\|^2 + \lambda \cdot \|w\|_1$ не гладкая (в точках, где $w_j = 0$, производная не определена). В таких случаях используются итеративные численные методы, например покоординатный спуск или градиентные методы.

Elastic Net — как смесь L_1 и L_2 — решается итеративно.

12. Другие решения вырожденности матрицы

Регуляризация — не единственный способ справиться с проблемой необратимости матрицы $X^\top X$.
В некоторых задачах регуляризация может быть избыточной, а в других — не подходить с точки зрения целей анализа (например, если важна интерпретируемость).

Рассмотрим другие способы решения проблемы вырожденности.

12.1. СЕЛЕКЦИЯ ПРИЗНАКОВ (FEATURE SELECTION)

Селекция признаков помогает убрать лишние, шумные или сильно коррелированные признаки.
Это нужно:

- чтобы устранить мультиколлинеарность;
- повысить интерпретируемость;
- ускорить обучение и уменьшить объём используемой памяти.

Подробно методы селекции признаков рассмотрим на 12-й неделе.

12.2. УВЕЛИЧЕНИЕ ОБЪЁМА ВЫБОРКИ (INCREASING SAMPLE SIZE)

Проблема. Когда число признаков d приближается к числу объектов ℓ или превосходит его, матрица $X^\top X$ теряет полный ранг. Тогда решение линейной регрессии становится нестабильным или вообще перестаёт существовать.

Решения

- Сбор дополнительных данных:
  - расширить базу новых наблюдений;
  - объединить несколько источников данных.
- Синтетическое увеличение выборки:
  - для изображений или текстов — повороты, шум, синонимы;
  - для табличных данных — SMOTE, bootstrapping.
- Генерация «псевдообъектов»:
  - приближённые выборки через генеративные модели (GAN, VAE);
  - имитация новых примеров на основе существующих.

12.3. ИСПОЛЬЗОВАНИЕ УСТОЙЧИВЫХ СТОХАСТИЧЕСКИХ АЛГОРИТМОВ

Вместо аналитического решения линейную регрессию можно обучать с помощью численных методов оптимизации, например градиентного спуска и его стохастических версий.

Такие методы не требуют явного обращения матрицы $X^\top \cdot X$, поэтому могут обучать модель даже в условиях вырожденности или высокой размерности.

Они работают с текущей оценкой коэффициентов и постепенно её улучшают, двигаясь в направлении антиградиента.

Такие методы особенно полезны, если:

- данных слишком много и аналитическое решение становится слишком дорогим вычислительно;
- матрица плохо обусловлена.

Градиентный спуск — один из основных методов оптимизации в машинном обучении. Далее разберём, как он работает для линейной регрессии.

13. Градиентный спуск для линейной регрессии

Аналитическое решение линейной регрессии — это точка минимума функции потерь, найденная напрямую.

Но при большой размерности данных или плохой обусловленности матрицы $X^\top X$ удобнее искать решение итерационно — с помощью градиентного спуска.

ОПРЕДЕЛЕНИЕ

Градиентный спуск — численный метод нахождения *локального* минимума или максимума функции с помощью движения вдоль градиента, один из основных численных методов современной оптимизации.

Для линейной регрессии градиентный спуск выглядит так.

- Функция потерь:

$$J(w) = \|X\,w - y\|^2 .$$

- Градиент функции потерь:

$$\nabla J(w) = 2\,X^{\mathsf{T}}\,(X\,w - y).$$

- Правило обновления. На каждом этапе делаем шаг в сторону антиградиента:

$$w \leftarrow w - \eta\,\nabla J(w),$$

где $\eta > 0$ — скорость обучения (англ. *learning rate*).

Алгоритм градиентного спуска:

1. Инициализировать $w^{(0)}$ (например, нуль-вектором).
2. Для $k = 1, 2, \ldots$ до сходимости:
   1) вычислить градиент:
   $$\nabla J\big(w^{(k-1)}\big);$$
   2) обновить параметры:
   $$w^{(k)} = w^{(k-1)} - \eta\,\nabla J\big(w^{(k-1)}\big).$$
3. Остановиться, когда изменения w или значение $J(w)$ становятся малы.

Градиентный спуск позволяет получать приближённое решение даже при очень большой размерности, когда прямая инверсия $\left(X^{\mathsf{T}}X\right)^{-1}$ слишком тяжела или невозможна.

Основной минус градиентного спуска — появляется гиперпараметр η (скорость обучения, англ. *learning rate*). Этот параметр регулирует размер шага обучения.

- Если у него слишком маленькое значение, то оптимизатор будет долго идти к оптимуму функции маленькими шагами.
- Если параметр очень большой, оптимизатор может не сойтись к оптимуму. Поэтому приходится подбирать промежуточное значение.

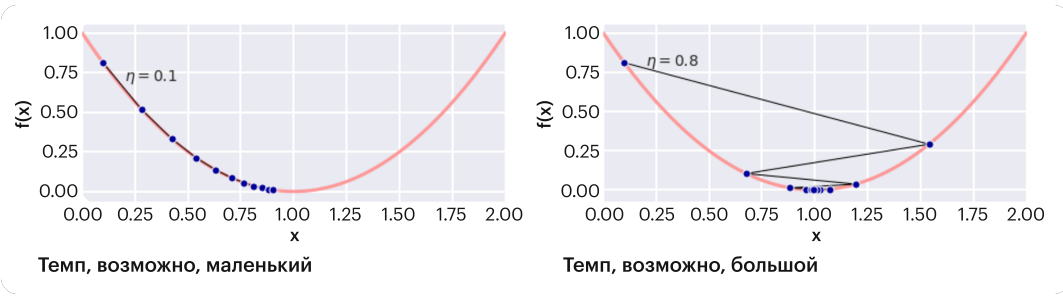


Рис. 23. Пример маленького и большого learning rate. Изображение создано автором

13.1. ПОЧЕМУ ГРАДИЕНТНЫЙ СПУСК НУЖЕН ДЛЯ БОЛЬШИХ ВЫБОРОК

При больших объёмах данных аналитическое решение становится слишком дорогим по времени и памяти. Нужно вычислить матрицу $X^\top X$, а затем обратить её, что плохо масштабируется на большие выборки. Градиентный спуск работает проще. На каждой итерации нужно:

- вычислить Xw (матрица на вектор);
- вычислить градиент (ещё одна матричная операция).

Это гораздо дешевле (сложность $O(nd)$, d — число признаков) и масштабируется на большие выборки. При этом точность решения можно контролировать через количество итераций.

13.2. ЕСТЬ ЛИ РИСК ПОПАСТЬ В ЛОКАЛЬНЫЙ МИНИМУМ

Напомним, что градиентный спуск может попасть в локальный минимум — не найти оптимальную точку.

Но для линейной регрессии это не проблема: функция потерь — выпуклая (квадратичная), и у неё только один глобальный минимум.

Если матрица $X^\top X$ вырождена, минимумов может быть несколько, но все они дают одинаковое значение функции потерь.

Поэтому при правильно выбранной скорости обучения градиентный спуск придёт к тому же решению, что и аналитический метод.

Иногда размерность данных настолько большая, что просто невозможно вычислить решение линейной регрессии аналитически (не хватит памяти), поэтому приходится идти небольшими шагами по части выборки и использовать градиентный спуск.

14. Реализации линейной регрессии: sklearn и statsmodels

В Python есть две основные библиотеки, которые позволяют строить линейную регрессию:

- `scikit-learn` (`sklearn`) — фокус на машинное обучение, быстрые предсказания, масштабируемость;
- `statsmodels` — фокус на статистику, гипотезы, доверительные интервалы и интерпретацию.

Каждая из них решает одну и ту же задачу — найти такие коэффициенты w, чтобы модель $\hat{y} = X \cdot w$ хорошо приближала значения y, но библиотеки делают это с разными приоритетами.

14.1. SCIKIT-LEARN: БЫСТРОЕ И ГИБКОЕ ОБУЧЕНИЕ МОДЕЛИ

`sklearn` ориентирована на прикладные ML-задачи. Основной класс:

```
from sklearn.linear_model import LinearRegression

model = LinearRegression()
model.fit(X, y)
```

Плюсы:

- удобный API (инициализируем модель → fit → predict);
- можно легко подставить в пайплайн или GridSearch;
- автоматически применяет устойчивые методы (например, SVD; об этом позже);
- подходит для больших данных, так как есть стохастическая реализация;
- есть регуляризованные версии — Ridge, LASSO, ElasticNet.

Минусы:

- по умолчанию нет статистической информации — p-значений, доверительных интервалов и стандартных ошибок;
- библиотека хуже подходит для статистического анализа и проверки гипотез.

> Если основная цель — предсказание и ML-задачи, обычно используют **sklearn**.

14.1.1. Как считается решение линейной регрессии в sklearn

В `sklearn` линейная регрессия реализована в классе `LinearRegression`. Он подбирает коэффициенты модели w, минимизируя среднеквадратичную ошибку:

$$\|Xw - y\|^2 \to \min_w.$$

Однако способ решения этой задачи может быть разным. В зависимости от структуры данных, размера выборки и требований к устойчивости внутри используются разные численные методы:

- SVD-разложение (сингулярное разложение, англ. *singular value decomposition*) — используется при работе с плотными матрицами, когда на вход подаётся матрица формата `np.array`.
- QR-разложение — разложение в ортогональную и верхнетреугольную матрицу. Используется при разреженной матрице данных (когда много нулей), на вход модели подаются данные в формате `scipy.sparse`. Такой формат данных в реальной практике можно применять при большом количестве категорий в данных, которые закодированы через OHE.

Метод выбирается автоматически в зависимости от формата входных данных — вручную настраивать это обычно не нужно.

14.1.2. SVD-разложение (сингулярное разложение)

Если матрица $X^\top X$ вырождена или плохо обусловлена, аналитическое решение через прямую инверсию работает нестабильно. В таких случаях используют SVD-разложение — метод, который позволяет устойчиво вычислить псевдообратную матрицу.

1. Вычисляем SVD матрицы X:
$$X = U \Sigma V^\top,$$
где:
   - где $U \in \mathbb{R}^{\ell \times \ell}$ — ортонормированная матрица левых сингулярных векторов;
   - $\Sigma \in \mathbb{R}^{\ell \times (d+1)}$ — диагональная матрица сингулярных чисел $\sigma_1 \geq \sigma_2 \geq \cdots$;
   - $V \in \mathbb{R}^{(d+1) \times (d+1)}$ — ортонормированная матрица правых сингулярных векторов.
2. Формируем матрицу «обратных» сингулярных чисел Σ^+:
   - по диагонали ставим $1/\sigma_i$ для всех $\sigma_i \neq 0$;
   - если $\sigma_i = 0$ (или очень близко к нулю), ставим вместо $1/\sigma_i$ значение 0, чтобы не делить на ноль.
3. Вычисляем псевдообратную X^+:
$$X^+ = V \Sigma^+ U^\top.$$
Это даёт наилучшую в смысле наименьшей нормы векторную оценку решения даже при вырожденности.
4. Получаем коэффициенты модели:
$$w = X^+ y.$$

Именно этот метод используется **по умолчанию** в `sklearn.LinearRegression`, так как он:

- стабилен к мультиколлинеарности;
- не требует инверсии $X^\top \cdot X$, что вычислительно трудно.

Код получения весов через SVD-разложение:

```
from numpy.linalg import svd, pinv

# Если хочется обратить вручную
U, s, VT = svd(X, full_matrices=False)
s_inv = np.diag(1 / s)
w = VT.T @ s_inv @ U.T @ y

# Если хочется воспользоваться одним реализованным методом
w = pinv(X) @ y
```

14.1.3. QR-разложение

Если аналитическое решение нестабильно, можно разложить матрицу X как произведение:

$$X = QR,$$

где:

- Q — ортонормированная матрица ($Q^\top Q = I$);
- R — верхнетреугольная матрица.

Тогда задача сводится к решению системы

$$\begin{cases} QRw = y, \\ Rw = Q^{\mathsf{T}} y. \end{cases}$$

Эта система решается с помощью метода обратной подстановки, который численно более стабилен и быстрее по сравнению с обращением матрицы.

Код получения весов модели через QR-разложение:

```
Q, R = np.linalg.qr(X_b)
w = scipy.linalg.solve_triangular(R, Q.T @ y)
```

14.1.4. Итеративные методы (градиентный спуск, SGD)

Когда данных много, хранить всю матрицу X может быть невозможно. В таких случаях применяют численные приближения, такие как стохастический градиентный спуск (SGD). Он шаг за шагом улучшает оценку w, используя только часть данных за раз.

SGD особенно полезен при:

- большом количестве данных, которые не помещаются целиком в память;
- большом количестве переменных, когда матрица $X^{\mathsf{T}}X$ плохо обусловлена;
- работе с потоками данных, если хотим менять веса модели онлайн.

В `sklearn` это реализовано в `SGDRegressor`. У него доступны следующие параметры:

```
from sklearn.linear_model import SGDRegressor

sgd = SGDRegressor(
    max_iter=1000,
    tol=1e-3,
    eta0=0.01,            # Начальная скорость обучения
    learning_rate='invscaling',  # Схема скорости обучения (
     ↪ {'constant','optimal','invscaling','adaptive'} )
    power_t=0.25,         # Экспонента для настройки шага обучения
    penalty='l2',         # L2-регуляризация (можно 'l1' или 'elasticnet')
    alpha=0.0001,         # Сила регуляризации
    average=False,        # Если True, возвращает усреднённый вектор весов, а не
     ↪ последние. Потенциально усреднение добавляет стабильности, если взять последние k
     ↪ шагов
    n_iter_no_change=5    # Для learning_rate='adaptive', уменьшает lr, если функция
     ↪ потерь не падает n шагов
)
sgd.fit(X, y)
```

Таким образом, `sklearn` предоставляет сразу несколько способов решения одной и той же задачи. Выбор метода зависит:

- от размерности и структуры данных;
- устойчивости $\left(X^\top X\right)^{-1}$;
- потребности в регуляризации;
- объёма памяти и скорости.

Рекомендации

1. Если матрица X не слишком большая (помещается в память) и не слишком плохо обусловлена, лучше применять аналитическое решение или Ridge / LASSO с SVD / QR (они более стабильны).
2. Когда n (число объектов) очень велико, а d (число признаков) невелико, лучше использовать SGD — памяти требуется $O(d)$, а не $O(nd)$.
3. Если новые объекты приходят постоянно, используй online SGD — каждый новый набор объектов обновляет w, но нужно контролировать, чтобы данные были качественными и мы не изменили веса слишком сильно.

14.2. STATSMODELS: БОЛЕЕ ГЛУБОКИЕ ВЫВОДЫ

Библиотеку `statsmodels` создавали как text-аналог популярных статистических пакетов из R (`lm`, `glm`, `plm`, `arima`).

В эконометрике R до сих пор используют чаще: новые статистические методы обычно сначала появляются там, а уже потом — в Python-библиотеках.

При этом базовые модели, включая линейную регрессию, в statsmodels давно реализованы и активно используются на практике.

Главное отличие `statsmodels` от `sklearn` — библиотека ориентирована не только на предсказание, но и на статистический анализ модели.

В `statsmodels` есть:

- подробный OLS-summary с p-значениями, доверительными интервалами и статистическими тестами;
- формульный интерфейс, похожий на синтаксис R;
- большое количество статистических моделей — `logit`, `probit`, модели временных рядов и панельных данных.

В `statsmodels` линейная регрессия называется OLS (Ordinary Least Squares) — метод наименьших квадратов (МНК). Название связано с тем, что модель минимизирует сумму квадратов ошибок предсказания.

14.2.1. Как работает OLS в statsmodels

Процесс построения модели линейной регрессии (OLS) обычно состоит из нескольких этапов.

1. **Ввод формулы** с помощью **формульного парсера** (`patsy`), который работает по аналогии с синтаксисом R. Или вводим матрицу данных, как в `sklearn`.
   Например, задаём линейную регрессию строкой вида

   ```
   y ~ x1 + x2 + C(cat) + I(x2**2) + x1:x2
   ```

   где:
   - `C` — категориальная переменная;
   - `I` — полиномиальные трансформации;
   - `x1:x2` — произведение x1 и x2. Если написать как `x1*x2`, то добавятся сразу три переменные (x1 и x2 по отдельности, а также их произведение).

   `Patsy` автоматически строит матрицу входных данных X.

2. **Оценка коэффициентов.**
   По умолчанию используется метод SVD, но также может применяться QR-разложение. В отличие от `sklearn`, в `statsmodels` способ решения выбирается автоматически как гиперпараметр. Разреженный формат `statsmodels` не поддерживает.

3. **Проверка зависимости целевой переменной от предикторов.**
   В случае многомерных предикторов сложно увидеть зависимость Y от X. Можно построить график Y от X_1, оставив только коэффициент при X_1 и свободный член. Но этот график окажется совсем не похожим на данные, поскольку $X_2, \dots, X_k$, вообще говоря, не ортогональны X_1.
   Более правильна следующая процедура, которую называют **Partial Plot**. Это график $\widetilde{Y}_i$ от $\widetilde{X}_i$, где $\widetilde{Y}_i$ получается удалением из Y его проекции на X_j при $j \neq i$, а $\widetilde{X}_i$ также строится по X_i. Иначе говоря, мы:
   - Строим регрессию Y по $X_{\cdot,j}$, $j \neq i$. Вычитаем из каждой точки Y линейный прогноз регрессии, получаем вектор $\widetilde{Y}$.
   - Строим регрессию $X_{\cdot,i}$ по $X_{\cdot,j}$, $j \neq i$. Вычитаем из каждой координаты $X_{\cdot,i}$ линейный прогноз регрессии, получаем вектор $\widetilde{X}_i$.
   - Строим линейную регрессию $\widetilde{Y}$ по $\widetilde{X}_i$ и строим двумерный набор точек и прямую на нём.

   В Python она есть в `statmodels.graphics.plot_partregress`.

4. **Оценка мультиколлинеарности.**
   Для того чтобы убрать переменные, линейно выражающиеся с помощью других, зачастую используют VIF — Variance Inflation Factor:

   $$\mathrm{VIF}_i = \frac{1}{1 - R_i^2},$$

   где R_i^2 — коэффициент детерминации для регрессии X_i по остальным переменным.
   Иначе говоря, мы считаем отношение суммы квадратов ошибки при прогнозе X_i константой и суммы квадратов ошибки при прогнозе X_i линейной комбинацией других признаков. Если отношение велико, то предиктор сильно зависим с остальными. VIF можно найти с помощью `variance_inflation_factor` из `statsmodels.stats.outliers_influence`. Обычно на практике стараются избегать величины с VIF больше 10 (или 5 в зависимости от запросов).
   Удаление коррелированных признаков может снизить мультиколлинеарность, но иногда ухудшает качество предсказания, поэтому здесь всегда приходится искать баланс.

5. **Вывод статистик:**
   - для каждого коэффициента рассчитывается **t-статистика**:

     $$t_j = \frac{\widehat{w}_j}{\mathrm{SE}(\widehat{w}_j)},$$

     где SE — стандартная ошибка, извлекаемая из оценки ковариационной матрицы $(X^\mathsf{T}X)^{-1}\hat{\sigma}^2$;
   - p**-value** по t-распределению с $n - p$ степенями свободы вычисляется на основе t-статистики;
   - **доверительный интервал**:

     $$\widehat{w}_j \pm t_{1-\alpha/2,\, n-p}\, \mathrm{SE}(\widehat{w}_j).$$

6. **Проверка гипотез и диагностические тесты.**
   - **F-тест** общей значимости модели сравнивает объяснённую дисперсию с остаточной (необъяснённой).
     Нулевая гипотеза теста H_0: $\vec{a} = 0$.
     При H_0 статистика имеет распределение Фишера:

     $$\frac{(n-p)\,R^2}{(1-R^2)(p-1)} \sim F_{p-1,\,n-p}.$$

   - **Jarque–Bera** — тест на нормальность ошибок.
     Критерий Харке–Бера применяется для проверки гипотезы о нормальности распределения и основывается на коэффициентах асимметрии Sk и эксцесса C. Он подменяет гипотезу о нормальности на гипотезы о том, что эксцесс и асимметрия распределения такие же, как и у нормального. Асимметрией и эксцессом называют

     $$\frac{\mu_3}{\mu_2^{3/2}}, \qquad \frac{\mu_4}{\mu_2^2} - 3,$$

     где μ_i — i-й центрированный момент. Для их оценки естественно использовать аналогичные формулы с $\widehat{\mu}_i = \overline{(X - \overline{X})^i}$:

     $$\mathrm{Sk} = \frac{\widehat{\mu}_3}{\widehat{\mu}_2^{3/2}}, \qquad \mathrm{K} = \frac{\widehat{\mu}_4}{\widehat{\mu}_2^2} - 3.$$

   - Для проверки нормальности остатков регрессионной модели рассматривается статистика

     $$\frac{n-p}{6}\left(S^2 + \frac{K^2}{4}\right).$$

     В Python тест реализован в `scipy.stats.jarque_bera(x)` (без учёта зависимости регрессоров). На практике зачастую игнорируется зависимость предикторов и используются классические критерии Шапиро–Уилка или D'Agostino.
   - **Breusch–Pagan** — тест на гомоскедастичность ошибок.

     ОПРЕДЕЛЕНИЕ

     Гомоскедастичность — однородность наблюдений, выраженная в постоянной дисперсии случайной ошибки. Формально, $\mathbb{E}(\epsilon^2 \mid X = x) = \mathrm{const}$.

     ОПРЕДЕЛЕНИЕ

     Гетероскедастичность — неоднородность наблюдений, выраженная в непостоянной дисперсии случайной ошибки.

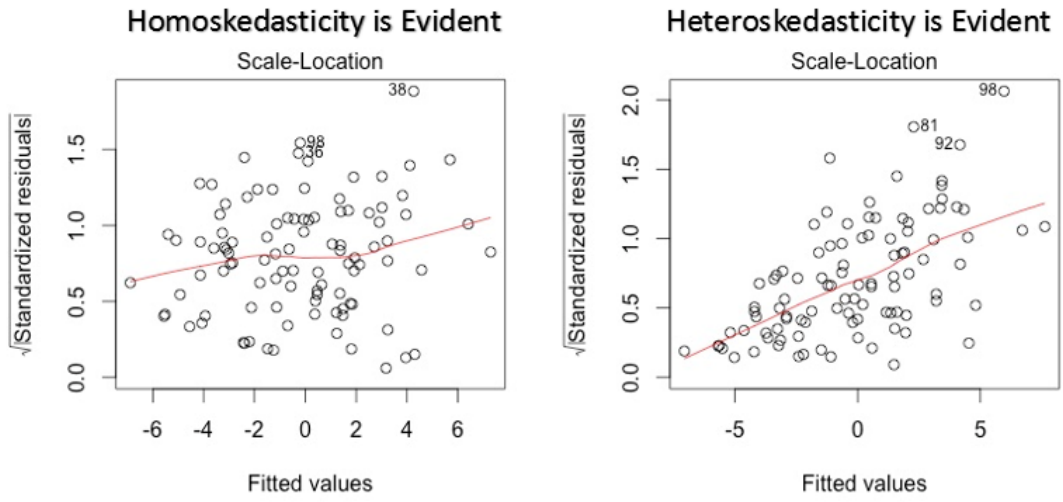


Рис. 24. Примеры гомоскедастичности и гетероскедастичности в данных. Источник

Для визуальной проверки строим `scale-location plot`, позволяющий оценить гомоскедастичность. Это график корня из стандартизированных остатков от прогноза зависимой переменной. Так на картинке справа мы видим, что разброс остатков увеличивается с ростом остатков. Этот график входит в то, что выдаёт `plot` от `fit`.

Критерий Breusch–Pagan предлагает построить вспомогательную регрессию квадратов остатков r_i^2 по признакам X_i, где $r_i = Y_i - \langle \tilde{w}, \tilde{X}_i \rangle$ — остаток исходной модели. Если дисперсия ошибки не зависит от признаков, такая регрессия почти ничего не объяснит и её R^2 будет близок к нулю. Статистика теста:

$$\mathrm{LM} = n\, R^2_{\text{вспом}},$$

где n — число наблюдений, а $R^2_{\text{вспом}}$ — коэффициент детерминации вспомогательной регрессии. При гомоскедастичности LM асимптотически имеет распределение χ^2 с числом степеней свободы, равным числу признаков во вспомогательной регрессии (без константы). Большое значение LM и малое p-значение говорят о гетероскедастичности. Тест улавливает только ту зависимость дисперсии от признаков, которую способна описать линейная вспомогательная регрессия: если связь существенно нелинейна, тест может её не обнаружить.

В Python реализация есть в функции `het_breuschpagan` в `statsmodels.stats.diagnostic`.

- **Durbin–Watson** — автокорреляция остатков, используется для временных рядов.

  Критерий **Durbin–Watson** — классическое решение. Он предлагает рассматривать статистику

  $$\mathrm{DW} = \left(\sum_{i=2}^{n} (r_i - r_{i-1})^2\right) \Big/ \left(\sum_{i=1}^{n} r_i^2\right).$$

  Эта статистика с ростом n сходится к $2 \cdot (1 - \rho)$, где ρ — коэффициент корреляции. Тем самым удаётся обнаружить ненулевую корреляцию ϵ_i и ϵ_{i+1}.

  Статистика Durbin–Watson считается функцией `durbin_watson` из `statsmodels.stats.stattools`; её значение выводится и в `results.summary()`.

  Одноимённая функция `dwtest` есть в R-пакете `lmtest`.

- **Влияние отдельных наблюдений (leverage, Cook's distance).**

  Рычаг (**leverage**) h_i для i-го наблюдения — это диагональный элемент матрицы $X(X^\top X)^{-1}X^\top$, соответствующий коэффициенту при y_i в прогнозе $\hat{y}_i$ (который представляет собой линейную функцию от y). У каждого наблюдения по оси x откладывается рычаг, а по оси y — стандартизированные остатки

  $$\frac{r_i}{\sqrt{(1 - h_i)\,\hat{\sigma}}}.$$

Эта стандартизация основана на том, что $\mathbb{D}r_i = \sigma^2(1 - h_i)$. Полученный график позволяет отследить отдельные значения, существенно влияющие на коэффициенты модели. На графике по каждой из переменных выделяются точки. На рис. 25 выделены соответствующие значения. Пунктирная линия на графике соответствует расстоянию Кука $D_i = 0{,}5$:

$$D_i = \frac{r_i^2 h_i}{k\hat{\sigma}^2 (1 - h_i)^2} = \frac{1}{k\hat{\sigma}^2} \sum_{j=1}^{n} \left(\hat{y}_j - \hat{y}_j^{(i)}\right)^2,$$

где $\hat{y}_j$ — прогноз, построенный по модели с исключённым наблюдением (x_j, y_j). Первая формула здесь удобна для подсчётов, а вторая отражает суть характеристики: расстояние Кука измеряет влияние i-го наблюдения.

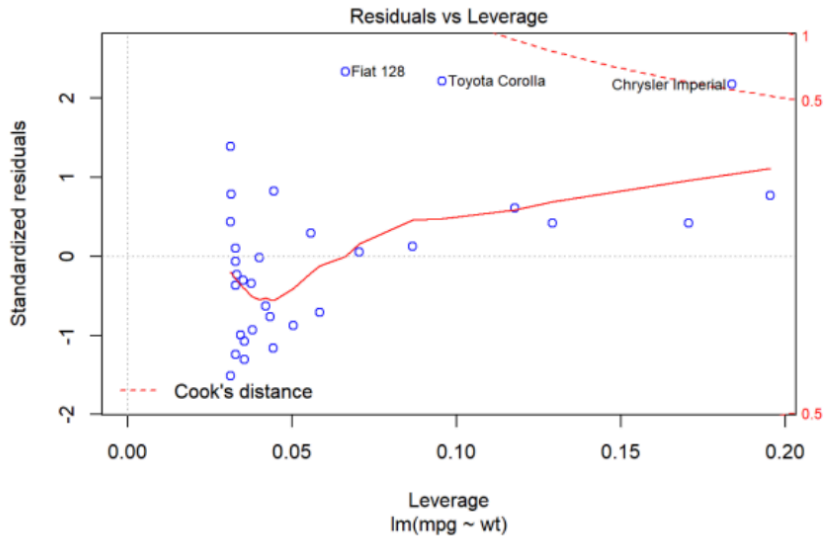


Рис. 25. График leverage с кривой на расстоянии Кука 0,5. Источник

Выделение точек на расстоянии свыше $0{,}5$ не слишком показательно, стоит в целом отсматривать наиболее влияющие точки. Использование расстояния Кука для оценки вклада наблюдений достаточно спорно. В качестве альтернативы можно рассмотреть предложенную Кимом меру

$$M_i = \frac{X_i \cdot (X^\top X)^{-2} \cdot X_i}{1 - h_i}.$$

Данные тесты нужны для того, чтобы проверить, насколько модель соответствует основным теоретическим предпосылкам линейной регрессии.

Преимущества `statsmodels`:

- статистическая строгость — содержит все необходимые инструменты для проверки статистических гипотез и проведения содержательного анализа;
- гибкость — легко расширяется пользовательскими тестами и моделями.

Если задача требует научного обоснования выводов, доверительных интервалов и формальных тестов, statsmodels станет незаменимым инструментом.

15. Пример реализации линейной регрессии в statsmodels

Рассмотрим, как можно пользоваться линейной регрессией в `statsmodels`, а также проанализируем результаты работы и выводы.

В `statsmodels` линейную регрессию можно задавать через прописную формулу с названием переменных:

```
# Если хотим использовать парсер формулы
import statsmodels.formula.api as smf

# Предполагаем DataFrame df с колонками y, x1, x2. Константа добавляется автоматически к признакам
model = smf.ols('y ~ x1 + x2', data=df)
results = model.fit()
```

Или заранее сформировать матрицу данных со всеми добавленными переменными и передать в модель (больше похоже на `sklearn`):

```
import statsmodels.api as sm

# Добавляем столбец единиц, чтобы добавить константу в регрессию.
X_const = sm.add_constant(X) # При таком подходе он не добавляется автоматически
model = sm.OLS(y, X_const)
results = model.fit()
```

Результаты работы модели можно рассмотреть с помощью метода `summary`:

```
print(results.summary())
```

Он возвращает развёрнутую таблицу, содержащую:

- значения коэффициентов;
- их стандартные ошибки;
- t-статистики и p-значения;
- доверительные интервалы;
- значение R^2 и скорректированное R^2;
- F-статистику всей модели.

Это важно:

- если нужно интерпретировать, какие признаки значимы;
- проверяются статистические гипотезы;
- результаты будут использоваться в научной публикации;
- важны интерпретации, а не просто точность предсказания.

```
                            OLS Regression Results
==============================================================================
Dep. Variable:                      y   R-squared:                       0.816
Model:                            OLS   Adj. R-squared:                  0.812
Method:                 Least Squares   F-statistic:                     214.6
Date:                Wed, 23 Apr 2025   Prob (F-statistic):           2.40e-36
Time:                        01:16:35   Log-Likelihood:                -136.08
No. Observations:                 100   AIC:                             278.2
Df Residuals:                      97   BIC:                             286.0
Df Model:                           2
Covariance Type:            nonrobust
==============================================================================
                 coef    std err          t      P>|t|      [0.025      0.975]
------------------------------------------------------------------------------
const          2.9486      0.097     30.479      0.000       2.757       3.141
x1             2.1375      0.104     20.563      0.000       1.931       2.344
x2            -1.0540      0.094    -11.173      0.000      -1.241      -0.867
==============================================================================
Omnibus:                        1.222   Durbin-Watson:                   2.397
Prob(Omnibus):                  0.543   Jarque-Bera (JB):                0.715
Skew:                          -0.127   Prob(JB):                        0.699
Kurtosis:                       3.327   Cond. No.                         1.64
==============================================================================
```

Рис. 26. Пример вывода результатов работы модели. Изображение создано автором

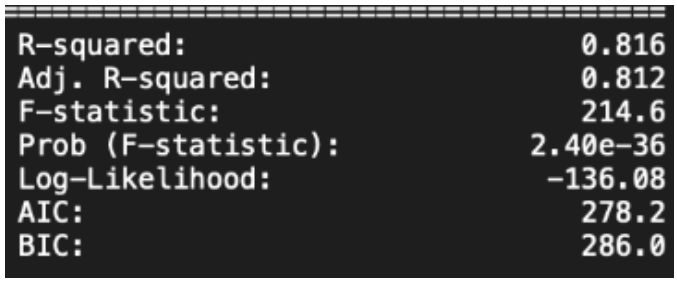
15.1. ВЕРХНЯЯ ЧАСТЬ ТАБЛИЦЫ: ОБЩАЯ ИНФОРМАЦИЯ

```
==========================================
Dep. Variable:                          y
Model:                                OLS
Method:                     Least Squares
Date:                    Wed, 23 Apr 2025
Time:                            01:16:35
No. Observations:                     100
Df Residuals:                          97
Df Model:                               2
Covariance Type:                nonrobust
==========================================
```

Рис. 27. Метаинформация из примера результатов работы модели. Изображение создано автором

- **Dep. Variable** — переменная, которую модель предсказывает (в нашем случае y).
- **No. Observations** — сколько строк в выборке (объектов).
- **Df Residuals** — число степеней свободы остатков ($n - d$, где n — количество наблюдений, d — число параметров).
- **Df Model** — число коэффициентов, не считая константы (если есть).
- **Covariance Type** — по умолчанию используется ковариация, которая неустойчива к гетероскедастичности ошибки. Чтобы воспользоваться устойчивой версией, необходимо передать в `fit` `cov_type=HC0/HC1/HC2/HC3`.

15.2. МЕТРИКИ МОДЕЛИ



R-squared:	0.816
Adj. R-squared:	0.812
F-statistic:	214.6
Prob (F-statistic):	2.40e-36
Log-Likelihood:	-136.08
AIC:	278.2
BIC:	286.0

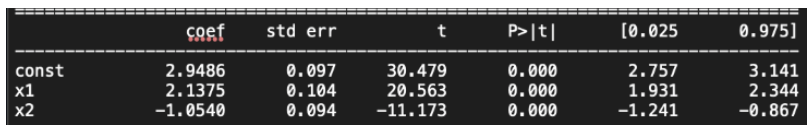
Рис. 28. Оценки качества из примера результатов работы модели. Изображение создано автором

- **R-squared** — доля дисперсии y, объяснённая моделью. Чем ближе к 1, тем лучше.
- **Adj. R-squared** — скорректированный R^2 (штрафует за добавление признаков; чем больше признаков, тем проще получить большой R^2, поэтому нужен штраф для контроля).
- **F-statistic** — проверка общей значимости модели — все w_j равны или не равны 0.
- **Prob (F-statistic)** — p-значение для F-статистики. Если оно $< 0{,}05$, модель в целом значима на 5%-м уровне (есть ещё $0{,}01\%$, 1%, 10%, но обычно смотрят на 5%).
- **Log-Likelihood** — правдоподобие, насколько хорошо данные описываются моделью.
- **AIC BIC** используется для сравнения моделей. Значение не несёт в себе никакой информации, его можно использовать как критерий для отбора модели, который определяет, насколько сильно модель переобучена. Чем меньше значение — тем лучше модель. BIC отличается от AIC тем, что он больше штрафует за дополнительные (и не влияющие ни на что) параметры модели.

$$\text{AIC} = 2d - 2\ln(L);$$
$$\text{BIC} = d\ln(n) - 2\ln(L).$$

Здесь:
  - L — значение функции правдоподобия;
  - n — количество наблюдений;
  - d — количество переменных.

15.3. ТАБЛИЦА КОЭФФИЦИЕНТОВ



	coef	std err	t	P>|t|	[0.025	0.975]
const	2.9486	0.097	30.479	0.000	2.757	3.141
x1	2.1375	0.104	20.563	0.000	1.931	2.344
x2	-1.0540	0.094	-11.173	0.000	-1.241	-0.867

Рис. 29. Результаты оценки коэффициентов линейной регрессии. Изображение создано автором

- `coef` — оценка коэффициента (например, $w_1 = 2{,}13$).
- `std err` — стандартная ошибка; показывает, насколько точно оценён коэффициент (предполагается, что коэффициент на самом деле лежит в интервале от $2{,}03$ до $2{,}23$).
- `t` — t-статистика, отношение коэффициента к его ошибке ($t =$ `coef/std err`).
- `P > |t|` — p-значение для t-статистики. Если $< 0{,}05$, признак статистически значим.
- `[0.025, 0.975]` — 95%-й доверительный интервал для значения коэффициента. Если он **не содержит** 0, признак тоже считается значимым.

15.4. ПРИМЕР ИНТЕРПРЕТАЦИИ СТРОКИ ДЛЯ ПРИЗНАКА x_1

Если рассмотреть полученные значения для признака x_1, то можно сделать вывод:

- что при прочих равных при увеличении x_1 на 1 единицу y в среднем увеличивается на $2{,}14$;
- этот эффект статистически значим ($p < 0{,}05$);
- мы уверены, что влияние x_1 неслучайно.

16. Какие библиотеки лучше использовать

16.1. SCIKIT-LEARN

Что делает:

- быстрые реализации линейных моделей (`OLS`, `Ridge`, `Lasso`, `ElasticNet`) и стохастического градиентного спуска для линейной регрессии (`SGDRegressor`);
- встроенные инструменты для пайплайнов, масштабирования, кросс-валидации и поиска гиперпараметров.

Когда использовать:

- нужно оперативно протестировать и подобрать гиперпараметры с помощью `GridSearchCV` / `RandomizedSearchCV`;
- готовим модель для продакшен-конвейера;
- работаем с большими или разреженными данными, онлайн-обучение.

16.2. STATSMODELS

Что делает:

- классическая статистическая оценка `OLS`, `GLS`, `RLM`, `MixedLM`;
- полный отчёт через метод `summary()` — коэффициенты, стандартные ошибки, p-значения;
- доверительные интервалы, AIC / BIC, тесты гетероскедастичности, нормальности остатков;
- гибкая настройка предположений об ошибках (гетероскедастичность, автокорреляция).

Когда использовать:

- эконометрика или социальные науки, где важна формальная проверка гипотез;
- требуется академическое качество отчёта (p-value, R^2 adj, F-тест);
- необходимо детально диагностировать модель и остатки.

17. Если кратко

→ Линейная регрессия — ML-модель, применимая при наличии гипотезы о линейной зависимости между целевой переменной и признаками.

→ Коэффициенты линейной регрессии могут быть вычислены несколькими способами:
- аналитически через нормальные уравнения, но могут возникнуть проблемы с обратимостью ковариационной матрицы;
- аналитически через SVD- или QR-разложения, но могут возникнуть проблемы при большом количестве данных;
- численными методами с помощью градиентного спуска.

→ Параметры модели подбираются минимизацией функции потерь — чаще всего MSE. Качество итоговой модели оценивают с помощью метрик:

Метрика	Формула	Интерпретация
MSE	$\frac{1}{n}\sum_{i=1}^{n}(y_i - \hat{y}_i)^2$	Средний квадрат ошибки; штрафует за крупные выбросы сильнее, чем MAE
RMSE	$\sqrt{\text{MSE}}$	То же, что MSE, но в единицах целевой переменной — проще интерпретировать
MAE	$\frac{1}{n}\sum_{i=1}^{n}\lvert y_i - \hat{y}_i\rvert$	Среднее абсолютное отклонение; менее чувствительна к выбросам
R^2	$1 - \frac{\sum(y_i - \hat{y}_i)^2}{\sum(y_i - \overline{y_i})^2}$	Доля дисперсии y, объяснённая моделью; от 0 до 1, чем выше — тем лучше

→ Повысить стабильность ответов линейной регрессии можно с помощью регуляризации, которая накладывает штраф на большие значения коэффициентов и смещает оптимум в сторону меньших весов. Регуляризация бывает:
- L_2 **(Ridge)** — штраф на квадрат коэффициентов; сжимает все веса, но не обнуляет.
- L_1 **(LASSO)** — штраф на абсолютные значения коэффициентов; часть весов зануляет, автоматически отбирая признаки.
- **Elastic Net** — взвешенная комбинация L_1 и L_2.

→ Две популярные реализации линейной регрессии находятся в пакетах:
- **scikit-learn** — удобен для быстрого написания пайплайнов и получения прогнозов.
- **statsmodels** — содержит инструменты для проверки статистических гипотез, что упрощает интерпретацию модели.